IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 19 June 2026, accepted 21 July 2026, date of publication 27 July 2026, date of current version 3 August 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3717031

RESEARCH ARTICLE

Edge-Responsive Local-Repair Particle Swarm Optimization for Dynamic Spatiotemporal Task Allocation of IoT-Connected UAV Swarms

YUTING SHEN[1], TIANYI HU[1], YI WAN[1], HAO SUN[2], AND MENGTING TAN[3,4]

[1]School of Environmental Science, Nanjing Xiaozhuang University, Nanjing 211171, China
[2]School of Computer Science and Engineering, Southeast University, Nanjing 211189, China
[3]Zhejiang Normal University, Jinhua 321004, China
[4]Beibu Gulf University, Qinzhou 535011, China

Corresponding author: Yi Wan (wan2721@163.com)

This work was supported in part by the Guangxi Science and Technology Major Program, China, under Grant AA24206003, and in part by the Key Research and Development Program of Guangxi, China, under Grant FN2504240008.

ABSTRACT Edge-enabled IoT missions rarely provide a fixed task set. Airborne sensors, ground devices, and local edge nodes continually report new targets, threat changes, and high-value functional-support opportunities after the initial plan has already committed a subset of UAVs to specific sequences. Replanning the whole mission on each arrival is too expensive to run at the tempo of operational events; ignoring arrivals throws away value. This paper attacks the online spatiotemporal task-allocation problem for heterogeneous UAV swarms carrying both mandatory and functional (support) tasks. We propose LRPSO-DTA, a three-module algorithm that combines a dynamic-awareness mechanism with a functional-task combination planner and an improved particle swarm optimiser that repairs only the affected, adjustable segment of the plan. Three design choices make the algorithm fast. i) sequence locking of short-horizon tasks prevents destructive rewrites of already-committed UAV actions, ii) a time-varying inertia and social-acceleration schedule shifts search from exploration to exploitation as iterations progress, and iii) a dependency-update operator touches only nodes and edges that a given arrival can possibly affect. Experiments across seven parameter sweeps, three scales, and six baselines show that LRPSO-DTA maintains an objective-value gap to CPLEX of 5.8% on average, cuts cumulative cost under successive arrival waves by roughly 60% relative to full replanning, and holds completion rate above 87% under high-burst-intensity arrivals where baselines drop below 75%. The local-repair design supports low-latency edge intelligence because it updates only the plan fragment exposed by the latest IoT event stream.

INDEX TERMS Dynamic task allocation, edge computing, Internet of Things, edge intelligence, particle swarm optimization, local repair, online replanning.

I. INTRODUCTION

A real unmanned aerial vehicle (UAV) mission does not sit still. A swarm that took off ten minutes ago with a fixed target list will, over the course of the next hour, meet new targets that were not on the list, lose visibility on others that moved or were destroyed, and pick up opportunities for functional support that did not exist at launch. Each of those events arrives as a small perturbation to the task set. Individually they are modest; collectively they are the norm rather than the exception.

The same pattern appears in edge computing and Internet of Things (IoT) systems. Sensing streams come from onboard cameras, radio modules, ground sensors, and mobile edge gateways, while connectivity and compute budgets vary during the mission. A controller that sends every update to a remote cloud optimiser may obtain a high-quality plan too late to execute it. A controller that performs local repair at the edge can instead react to IoT event streams while preserving the parts of the plan that are already committed.

The associate editor coordinating the review of this manuscript and approving it for publication was Muhammad Aleem.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

To make this concrete, consider a one-hour reconnaissance-and-strike mission beginning with 30 initial tasks known at launch. In our empirical study, three waves of arrivals typically appear over the course of that mission, each with around six new tasks and each splitting roughly three-to-one between new mandatory targets and new functional-support opportunities. By the time the last wave has landed, the swarm has executed perhaps a third of its originally planned tasks and committed to the next fraction; the remaining portion of the plan is still up for revision. Meanwhile, the dependency graph that described the launch-time mission no longer matches reality. New functional tasks have arrived that could shield existing mandatory tasks if added correctly; new mandatory tasks have arrived that the existing functional tasks might already be positioned to shield; and older functional tasks that have already been executed have turned into inactive edges. A planner that sees all these changes and updates only what is affected will be fast; a planner that rebuilds from scratch will be slow. A planner that doesn't see the changes at all will fail.

Two easy ways to deal with this are both bad. The first is to ignore the perturbations and fly the launch plan to completion; this guarantees no chaos but also guarantees that late-arriving high-value targets are missed and that support tasks are allocated according to an outdated map. The second is to replan the whole mission on every arrival. This captures every new opportunity, but pays for it with enormous compute and with instability. Every replan tends to shuffle UAV routes that were about to execute, which in practice is even worse than missing a late target. The right answer sits between the two. Touch only what needs to be touched, and touch it in a way that propagates correctly through the spatial and temporal dependencies that were already established in the plan.

The two companion papers of this work have already settled the spatial and temporal dimensions of the same underlying problem. The spatial paper introduced a battle-field dependency graph with distance-decayed edges and an exponential risk-diffusion model; the temporal paper layered time-window activation indicators and a consensus-based bundle auction on top of that model. The missing piece is the *dynamic* dimension, where the plan evolves in response to arrivals. We attack that piece here.

A. DYNAMICS VERSUS STATICS. WHAT CHANGES

Many readers will have a mental model of static task allocation and wonder which parts of it survive in the dynamic setting. The short answer is. The cost model, the constraint structure, and the objective form survive. What changes is the decision cadence and the information available at each decision point.

In static allocation, the allocator makes one decision at $t = 0$ with full information about the task set. In dynamic allocation, the allocator makes a sequence of decisions $\pi(0), \pi(\tau_1), \ldots$ with the information at each decision point being the cumulative arrivals up to that point and the current UAV states. Each decision is conditioned on the current state; the allocator cannot anticipate future arrivals. This is reactive, not predictive, planning.

Two consequences follow. First, the *objective* of each decision is no longer the offline-optimal plan, which is not available without future knowledge, but the best plan consistent with current information. Second, each decision must respect the *history* of previous decisions, which is captured in our formulation by the locked prefix of the current plan.

B. CUMULATIVE COST MATTERS MORE THAN PER-ARRIVAL COST

A subtle but important point is that evaluating a dynamic planner by its per-arrival objective misses the dominant effect. If the planner thrashes, rewrites imminent tasks on each arrival, the per-arrival objective can be very good (the planner finds locally-optimal plans each time) while the cumulative cost over the mission is catastrophic (because the rewrites incur additional flight cost and risk exposure that accumulate). The metric that actually matters is the cumulative cost summed over the mission, which includes the per-decision cost and the rework cost.

LRPSO-DTA's design explicitly optimises for cumulative cost rather than per-arrival cost, by locking the prefix and tolerating sub-optimal per-arrival decisions in exchange for reduced rework. Our experiments validate this trade-off. LRPSO-DTA's per-arrival objective is sometimes slightly worse than a full replanner's, but its cumulative cost is consistently much lower.

1) PROBLEM SKETCH

A heterogeneous UAV swarm $\mathcal{U} = \{u_m\}$ executes an initial task set $\mathcal{T}_{\text{init}}$ starting at $t = 0$. Over the course of the mission, new tasks $\mathcal{T}_{\text{pop}}(t)$ arrive at times determined by an arrival process; each arrival is either mandatory or functional. The controller must, at each arrival, update the plan so that (i) cumulative completion rate stays high, (ii) cumulative cost stays low, and (iii) computational latency per decision stays below an operationally acceptable bound. The utility is the same composite objective used in the spatial and temporal companion papers, carried over to the dynamic setting.

2) WHAT PRIOR WORK DOES NOT DO

Recent surveys and primary work on dynamic multi-UAV task allocation [1], [2], [3], [4], [5], [6] have converged on a small set of patterns: contract-net reallocation [7], [8], partial CBBA replanning [9], [10], online team competition [11], bionic metaheuristics [12], [13], and deep reinforcement learning [14], [15]. None of these, to our reading, simultaneously (a) models the spatial coupling between functional and mandatory tasks explicitly, (b) activates support only when temporal ordering is respected, and (c) repairs only the segment of the plan that a given arrival can actually influence. LRPSO-DTA is designed exactly to fill that three-way gap.

3) THE APPROACH

LRPSO-DTA is a three-module pipeline. The Dynamic Graph-Information update (DGI) module receives an arrival and touches only the local subgraph that the arrival can reach through distance-decayed edges or through temporal windows. The Scored Functional-task combination Planner (SFP) selects a high-value subset of the arrival to inject as a group. The Adaptive PSO with local repair (APSO) replans the *free* suffix of the plan, the portion that has not been committed to imminent execution, while leaving a locked prefix untouched.

Each of these modules carries a design choice. DGI uses a spatial index and a bounded-radius sweep so the update cost is sublinear in the total task count. SFP uses a max-heap with lazy evaluation so that benefit estimates are recomputed only for the single most promising candidate each round. APSO uses a time-varying inertia and social-acceleration schedule to move the swarm from exploration to exploitation smoothly, a two-stage bidding procedure to generate feasibility-preserving particles, and a repair operator to put invalidated particles back on the feasibility manifold.

4) RESULTS SUMMARY

Across seven parameter sweeps, three scales, and six baselines (CPLEX as upper bound, MOPSO, RISE, CBBA-Replan, GA-Online, Greedy-Insert), LRPSO-DTA maintains an average objective-value gap to CPLEX of 5.8%, cuts cumulative cost by roughly 60% relative to full replanning when three successive waves of arrivals hit, and keeps completion rate at 87% to 91% in high-burst-intensity regimes where CBBA-Replan drops to 76%. Per- arrival repair latency stays in the 48 to 56 ms range, which is comfortably within typical operational tempos.

We summarise the main contributions of this paper as follows.

- A dynamic-aware local-repair framework that only touches what a given arrival can influence, freezing short-horizon committed tasks against spurious rewrites.
- A functional-task combination planner that injects the high-value subset of a burst as a group, honouring the spatial coverage dependencies already established.
- An improved PSO with time-varying parameters and a repair operator, empirically within 5.8% of CPLEX on average while running orders of magnitude faster.

C. INTUITION BEFORE FORMALISM

For a first-time reader, the formal development in the next three sections can be dense. This subsection previews the main ideas in plain language so that the formalism later can be read as filling in details rather than introducing new concepts.

Idea 1, The dependency graph is always up to date. DGI keeps a live graph of which functional tasks can support which mandatory tasks, updated incrementally on each arrival. The graph is not rebuilt; it is edited. Rebuilding would cost $O(N^2)$; editing costs $O(\deg)$.

Idea 2, Admit functional arrivals as a screened group. When multiple functional tasks arrive together, their effects overlap. SFP ranks them by benefit-over-cost and admits in order with diminishing returns, ensuring no redundant supporters are admitted just because they happen to land in the same burst.

Idea 3, Only replan what you have to. APSO's PSO operates on the free suffix of the plan, not the whole plan. Tasks close to execution are locked and untouched. This keeps the per-arrival search small, fast, and stable.

Idea 4, Search schedule matters. The PSO parameters vary with iteration to shift from exploration (early iterations, high inertia) to exploitation (late iterations, high social pull). Fixed parameters wastefully explore at the end or under- explore at the beginning.

The rest of the paper is organised as follows. Section II reviews the dynamic-allocation literature. Section III formalises the dynamic spatiotemporal model. Section IV develops the three modules. Section V reports experiments. Section VI concludes.

II. RELATED WORK

We split the literature into four threads. (i) dynamic and online task allocation; (ii) event-triggered and rolling-horizon scheduling; (iii) metaheuristics for dynamic allocation, with an emphasis on PSO; (iv) functional and support tasks in dynamic settings.

A. ORIENTATION

Before diving into the threads, a brief map of the landscape helps. Dynamic multi-UAV task allocation has roughly three flavours in the literature. (a) *contract-net-style*, where incoming tasks are auctioned to UAVs individually; (b) *replan-on-arrival*, where the full plan is recomputed on each event; and (c) *incremental*, where only the affected portion of the plan is updated. Our LRPSO-DTA is in the incremental family but combines features from the other two. It admits arrivals through a screening step reminiscent of a contract net, and it uses PSO as a metaheuristic reminiscent of replan-style global search. The innovation is the combination and the scope limitation, not any single ingredient.

B. DYNAMIC AND ONLINE TASK ALLOCATION

Lerman et al. [16] laid groundwork for analysing dynamic multi-robot allocation; Parker's ALLIANCE architecture [17] is a classical fault-tolerant multirobot cooperation framework that still appears in modern references. Farinelli et al. [18] study distributed on-line assignment for multi-robot patrolling. Wei et al. [19] target dynamic search and retrieval. ElGibreen and Youcef-Toumi [20] focus on uncertain heterogeneous multi-agent environments. Choudhury et al. [21] attack dynamic allocation under uncertainty and temporal constraints. Ghassemi and Chowdhury [22] target disaster response with dynamic deadlines and payload constraints. Jin et al. [11] use a team- competition model. O'Brien et al. [23] target subterranean exploration. Shan et al. [24] handle

time-constrained collective transport. More recent UAV-focused works include Zhang et al. [4] on battlefield formations, Han et al. [25] on jamming under communication failures, Calvo and Capitán [26] on long-endurance missions, Zhao et al. [27] on throughput-maximising multi-UAV scheduling, Hai et al. [5] on real-time replanning, Zhang et al. [28] on persistent monitoring, and Alqefari and Menai [1], [6] who survey the field and propose a hybrid method. Tian et al. [29] propose an incremental task-replanning algorithm (ITRA) based on centralised-distributed negotiation; Sheng et al. [30] target resilience under destruction and constrained communication. None of these works combines a local subgraph update with a locked-prefix adaptive PSO in the way we do here.

C. SPECIFICALLY ON FUNCTIONAL-TASK DYNAMICS

Between the general dynamic allocation literature and the specific functional-task literature lies a small but growing intersection. Yan and Di's 2024 paper on optional tasks with hidden workload [31] adds a dynamic element by allowing the optional tasks to be admitted or rejected during planning; this is not exactly online admission but is a step in that direction. Ye et al. [32], [33] handle heterogeneous UAVs in dynamic environments with task-coupling constraints, which includes a form of dependency but not the spatial-temporal diffusion model we use. Wang et al. [8], [34] address reconnaissance task reassignment in dynamic environments, which is conceptually close to our problem but without the formal dependency graph. Sheng et al. [30] target resilience in dynamic task replanning under individual destruction and communication constraints, which is a broader but related problem. None of these works combines the three key features of LRPSO-DTA. a functional-task-aware dependency model, a local repair strategy with prefix locking, and an adaptive metaheuristic search. Our contribution is the combination.

D. EVENT-TRIGGERED AND ROLLING-HORIZON SCHEDULING

Buckman et al. [10] introduce partial replanning for decentralised dynamic task allocation; Chen et al. [9] combine CBBA with local replanning for time-sensitive environments. Johnson et al. [35] make CBBA asynchronous. Fei et al. [36] propose a rolling-inspired scheduler for emergency tasks. Han and Guo [13] use adaptive PSO within a rolling-horizon framework. Wang et al. [8] propose a contract-net two-stage assignment for reconnaissance reassignment. Hwang et al. [37] take a constraint-table centralised stance. These works share the rolling-horizon intuition with LRPSO-DTA but do not enforce a *locked prefix* that protects imminent- execution tasks from rewrite.

E. PSO AND METAHEURISTICS FOR DYNAMIC ALLOCATION

PSO is one of the most-used search strategies for combinatorial allocation, with extensive variants. Yafei and Liang [38] propose an improved multi-objective PSO based on area division for UAV task assignment. Wang et al. [39] attack multi-objective reconnaissance task allocation through dynamic PSO. Han and Guo [13] combine adaptive PSO with a rolling-horizon framework for energy- aware UAV task and path planning. Genetic algorithms remain competitive. Yan et al. [40] handle simultaneous arrival and resource constraints, and Zhao et al. [41] use waitable path coordination with modified GA for target-precedence-constrained multiple task assignment. Huang et al. [42] propose 4DI-GWO for heterogeneous UAVs in uncertain environments. Liu et al. [14] use deep reinforcement learning; Yu et al. [15] propose event-triggered graph reinforcement learning for dynamic UAV assignment under weak communication. Shi et al. [12] and Li et al. [43] propose bionic and deadlock-free schemes. LRPSO-DTA is closest in spirit to [13] and [39] but goes further by coupling the PSO to a dependency-graph update and a functional-task combination planner.

F. RESILIENCE AND FAULT TOLERANCE IN DYNAMIC SWARMS

A thread of work addresses resilience under faults and disturbances, which shares some concerns with dynamic arrivals. Parker's ALLIANCE [17] remains a classical reference for fault-tolerant multi-robot cooperation. Notomista et al. [44] provide a resilient and energy-aware task allocation framework in IEEE TRO. Sheng et al. [30] target resilience under complex terrain and destruction. Han et al. [25] address distributed jamming under communication failures. Our locked-prefix mechanism has a resilience flavour, it protects committed tasks from adversarial arrivals, but it is not designed for full fault tolerance (we assume UAVs do not fail mid-mission). A full resilience extension would combine LRPSO-DTA's dynamic repair with the fault-handling mechanisms of these works.

G. FUNCTIONAL / SUPPORT TASKS IN DYNAMIC SETTINGS

Support-type tasks in dynamic environments are the least-explored of our four threads. Yan and Di [45] introduce functional tasks but do not evolve them online; their hyper-heuristic is a static solver. Lv et al. [46] develop adaptive distributed observers for nonlinear leader-following multiagent systems, a complementary state-estimation and control capability, but do not address online task allocation. Yan et al. [31] add optional tasks with hidden workload, but still solve a static allocation. Ye et al. [32], [33] address heterogeneous UAVs in dynamic environments and with task-coupling constraints, but do not explicitly track the benefit and cost trade-off as arrivals change the graph. Deng et al. [47] allocate reconnaissance and strike tasks distributively. Peng et al. [2], [3] review and model dynamic allocation for ground targets under heterogeneous UAV swarms. Ponda et al. [48] consider dynamic communication

constraints. Our work targets the intersection of functional tasks and dynamic arrivals directly, and treats the two as a single coupled optimisation rather than a sequence of separate passes.

H. WHERE THE GAP IS

Prior dynamic allocation work either treats arrivals as independent items (contract-net / partial CBBA), pays for every arrival with a full metaheuristic restart (GA-Online, MOPSO), or ignores the spatial-temporal coupling that functional tasks induce (greedy insert). LRPSO-DTA is the first dynamic allocator we know of that combines. (i) a local dependency-graph update, (ii) a group-based functional-task planner, and (iii) an adaptive PSO that repairs only the free suffix of the plan.

A common objection to metaheuristic approaches such as PSO in time-critical settings is that the wall-clock cost is too high. This objection is well-founded when PSO is applied to the full planning problem. At $N = 180$ tasks, naive PSO can take several minutes per invocation, which is unacceptable under arrival periods of tens of seconds. The objection loses its force when PSO is applied only to the free suffix of the plan, which typically contains $K \ll N$ tasks. The effective search space is combinatorially smaller, and a population of $N_p = 30$ particles for $I_{\max} = 50$ iterations produces high-quality local repairs in tens of milliseconds per arrival, as our experiments confirm. Thus the apparent tension between "PSO is slow" and "online planning is fast" is resolved by constraining PSO to the right scope rather than by choosing a different metaheuristic.

III. PRELIMINARIES AND DYNAMIC MODEL

Figure 1 gives an overview of the system model: edge/IoT sources that generate dynamic task arrivals, the heterogeneous UAV swarm and its dynamic dependency graph, and the locked-prefix/free-suffix plan structure that the rest of this section formalises.

A. NOTATION

We reuse the notation of the spatial and temporal companion papers: UAVs $\mathcal{U}$, task set $\mathcal{T}(t) = \mathcal{T}_c(t) \cup \mathcal{T}_f(t)$, battlefield dependency graph $G_T(t)$ with distance-decayed edges w_{ji}, risk coefficient α, discount γ, depth H. Task properties. Position $\mathbf{p}_i$, base risk R_i, payload demand q_i, time window $[e_i, l_i]$, service duration s_i; for functional tasks, radius r_i and peak strength ω_i. The plan at time t is $\pi(t) = \{S_m(t)\}$.

Table 1 adds the dynamic-specific symbols used in this paper.

We now state the assumptions of the dynamic setting. The dynamic setting inherits the spatial and temporal assumptions of the companion papers and adds five of its own. (A1) Arrivals are observed instantaneously by the planner, without sensing or communication delay. (A2) The controller does not have access to a predictive model of future arrivals; decisions are reactive. (A3) Locked tasks commit fully: the UAV physically cannot re-route before τ_{lock} time units have elapsed after the lock. (A4) Arrivals do not cancel or remove existing tasks; the task set only grows. (A5) The launch-time plan $\pi(0)$ is produced by a static allocator (we use FTRDA, the spatial companion); the dynamic repair starts from there and does not rebuild the initial solution from scratch.

TABLE 1. Dynamic-specific notation (static symbols inherited from the spatial / temporal companion papers).

Symbol	Domain	Meaning
$\mathcal{T}_{\text{init}}$	set	tasks known at $t = 0$
$\mathcal{T}_{\text{pop}}(t)$	set	arriving tasks up to time t
τ_k	$\mathbb{R}_+$	time of k-th arrival
n_{τ_k}	$\mathbb{N}$	burst size at τ_k
$G_T(t)$	graph	dependency graph at time t
$\mathcal{N}_i^S(t)$	set	active supporters of t_i at t
τ_{lock}	$\mathbb{R}_+$	lock horizon (prefix freeze)
σ	sequence	free suffix of current plan
$M(t), F(t)$	sets	candidate pools (mandatory, functional)
$F^*(t)$	subset	SFP-selected functional injection
ν_i	$[0, 1]$	residual support demand on t_i
Θ_j	$\mathbb{R}$	SFP composite score of t_j
N_p	$\mathbb{N}$	PSO population size
$I_{\max}$	$\mathbb{N}$	PSO iteration cap
$w(k), c_1(k), c_2(k)$	$\mathbb{R}_+$	time-varying PSO coefficients
μ	$\mathbb{R}_+$	shape exponent for inertia schedule
s	$[0, 1]$	normalised PSO iteration $k/I_{\max}$

B. ARRIVAL MODEL

Arrivals $\mathcal{T}_{\text{pop}}(t)$ occur at times $\{\tau_1, \tau_2, \dots\}$. At each τ_k, a burst of n_{τ_k} tasks arrives; each is either mandatory or functional, with its own position, risk, window, and payload. We do not assume a probabilistic model of arrivals; the algorithm is reactive and does not exploit future knowledge.

C. DYNAMIC DEPENDENCY GRAPH

When a new task arrives, three kinds of changes can occur in $G_T(t)$. New nodes are inserted, new edges are induced between the new node and existing tasks within action radius, and weights on existing edges may shift (e.g., if a task completes). We keep a spatial index to localise these changes; the cost of an update is $O(\deg)$ where deg is the local degree, independent of the total graph size.

D. EFFECTIVE RISK AND OBJECTIVE

The effective risk $\widetilde{R}_i(t)$ is the same exponential compression as in the spatial paper, specialised to the current graph:

$$\widetilde{R}_i(t) = R_i^0 \cdot \exp\Big(-\alpha \sum_{j \in \mathcal{N}_i^S(t)} \phi_{ji}(t) \Big), \tag{1}$$

where $\mathcal{N}_i^S(t)$ is the set of actively supporting functional tasks (spatial edge, executed, and temporally activated as in the temporal paper). The objective is the same composite $f(\pi) = -\lambda_1 \mathcal{C}(\pi) + \lambda_2 \mathcal{P}(\pi)$ with $\lambda_1 = 0.05$, $\lambda_2 = 0.95$.

E. DECISION PROBLEM

At each arrival τ_k, the controller selects a new plan $\pi(\tau_k^+)$ that (i) respects all constraints of the spatial and temporal

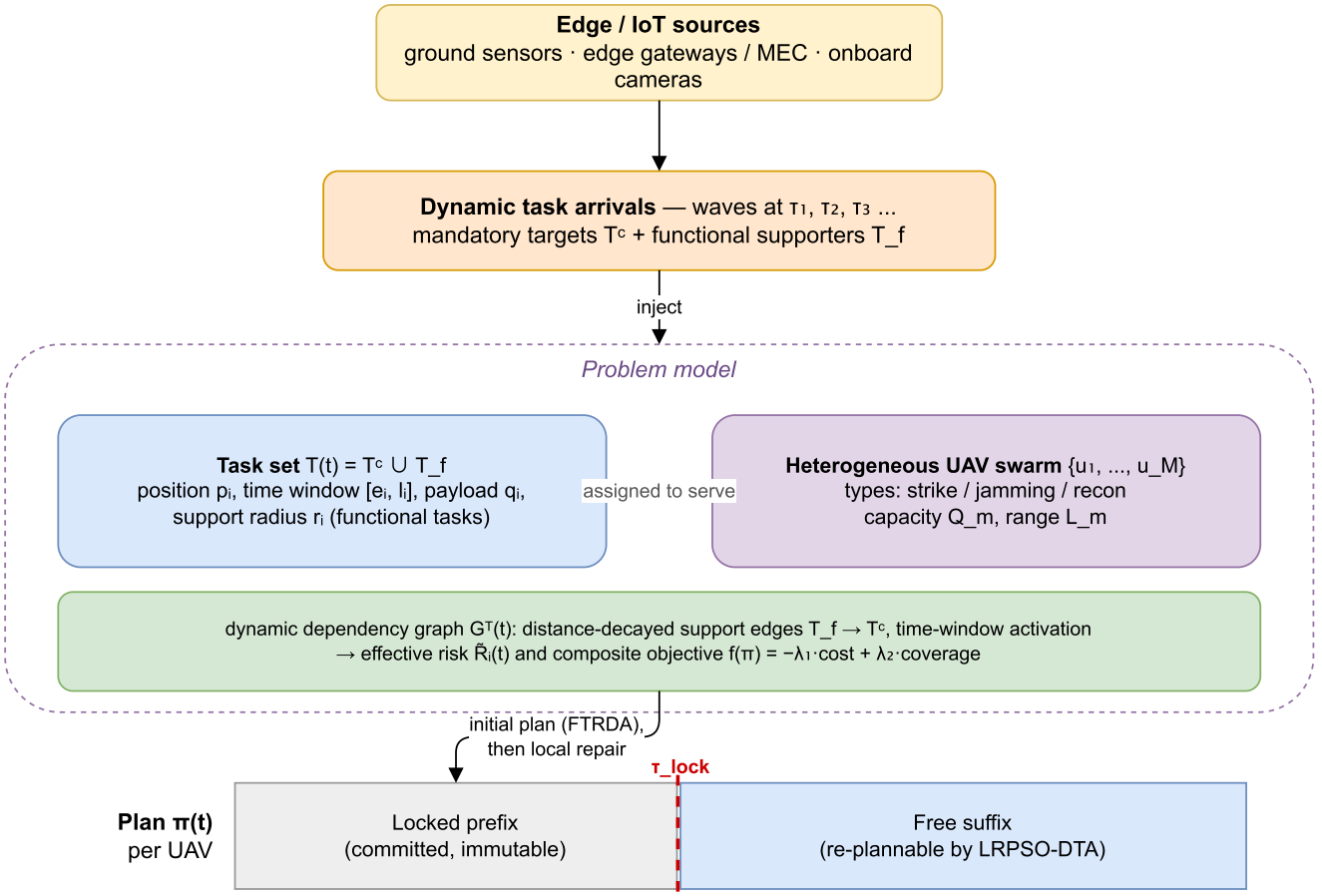


FIGURE 1. System model. Edge/IoT sources generate dynamic task arrivals (waves) carrying both mandatory targets and functional supporters. A heterogeneous UAV swarm serves these tasks under a dynamic dependency graph $G_T(t)$ with distance-decayed support edges and time windows. On each arrival the current plan is split into a locked prefix (committed, immutable) and a free suffix that LRPSO-DTA repairs.

companion problems, (ii) preserves the *locked prefix*, tasks scheduled to finish within τ_{lock} time units from τ_k are immutable, and (iii) maximises $f(\pi)$. This is the local repair problem.

1) TWO COMPETING PRESSURES

The dynamic planner faces two competing pressures. *responsiveness* (how quickly arrivals are incorporated) and *stability* (how reliably executing tasks are not disrupted). Full replanning is maximally responsive but minimally stable. Every arrival can shuffle every task, including imminent ones. Static planning with arrival buffers is maximally stable but minimally responsive. Arrivals wait for the next planning horizon. LRPSO-DTA sits between the two by locking only the near-term prefix and re-planning the rest. The lock horizon τ_{lock} is the single parameter that controls the trade-off. Long horizons favour stability, short horizons favour responsiveness.

2) FORMAL DEFINITION OF THE FREE SUFFIX

Let $\pi(\tau_k^-) = \{S_m\}$ denote the plan just before the arrival. For each UAV u_m, the locked prefix is the longest initial sub-sequence $L_m = (t_{m1}, \ldots, t_{m\ell_m})$ such that $t^e_{m\ell_m} \le \tau_k + \tau_{\text{lock}}$, that is, all tasks in L_m finish within the lock horizon. The free suffix is the remainder $F_m = S_m \setminus L_m$. Write $\sigma = \bigcup_m F_m$ for the cross-UAV free suffix set; the decision problem restricts to plans whose locked prefixes are unchanged, i.e., to reassignments and reorderings over σ together with any admitted arrivals.

3) WHY PREFIX LOCKING?

Rewriting a UAV's imminent task can cost more than the benefit of the rewrite. Consider a UAV that is 4 seconds away from executing a mandatory strike; an arrival suggests a re-route that would ultimately save 8 seconds of flight time across the rest of the mission. If re-routing requires turning the UAV around, the cost of the turn (fuel, time, risk of exposure during the turn) can easily exceed the 8 seconds of savings. Locking the prefix avoids this class of churn entirely at the price of occasionally missing small improvements. Empirically, the ablation in Table 6 shows that removing prefix locking *worsens* cumulative cost by 22%, which is striking because one might naively expect more freedom to help.

4) OBJECTIVE OVER SUCCESSIVE ARRIVALS

Over the mission lifetime, we track the cumulative objective

$$F_{\text{cum}} = \sum_{k} f(\pi(\tau_k^+)), \tag{2}$$

which captures both the per-arrival quality and the stability of the plan across arrivals. A planner with wildly oscillating per-arrival objectives will have lower cumulative objective than one with steady improvements, even if its best per-arrival objective is higher.

Theorem 1: The dynamic spatiotemporal functional task allocation problem is NP-hard even with a single arrival and a single UAV.

Sketch: Instantiate a single UAV, no functional tasks, a single arrival of k mandatory tasks with distinct positions and windows. The resulting problem is TSPTW, which is NP-hard. Adding more UAVs, functional tasks, or multiple arrivals can only make it harder. □

F. STRUCTURAL PROPERTIES

Three lemmas about the repair process anchor the design of LRPSO-DTA.

Lemma 1 (Prefix-Locking Preserves Feasibility): If the plan $\pi(\tau_k^-)$ is feasible immediately before an arrival, and the repair process only modifies the free suffix σ, then the post-repair plan $\pi(\tau_k^+)$ is feasible with respect to all locked-prefix constraints.

Proof Sketch: Locked tasks retain their original positions and timing by construction; their feasibility therefore carries over unchanged. Free-suffix modifications may introduce feasibility issues *within* the suffix, which the repair operator in APSO explicitly corrects. The combination guarantees overall feasibility. □

Lemma 2 (Incremental BDG Update is Exact): DGI's incremental update of $G_T(t)$ produces the same graph as rebuilding from scratch. No edge is missed and no spurious edge is introduced.

Proof Sketch: For an inserted task t_{new}, the set of new edges is exactly $\{(j,i) : j \in \mathcal{T}_f, i = t_{\text{new}}, \|t_j - t_{\text{new}}\| \le r_j\}$ or the symmetric set if t_{new} is functional. The spatial index returns exactly the relevant neighbours. Existing edges are unaffected by the insertion. For a completed task, outgoing edges are deactivated but the vertex remains in the graph (to preserve cascaded history); the deactivation is idempotent. Therefore the incremental graph equals the freshly built graph. □

Lemma 3 (SFP is a Bounded-Approximation Admission): Consider the aggregate benefit function $B(S) = \sum_{i\in\mathcal{T}_c(t)} R_i^0(1 - \exp(-\alpha \sum_{j\in S} w_{ji}\phi_{ji}))$ over functional subsets S. B is monotone non-decreasing and submodular. Greedy selection by $\Theta_j =$ marginal/cost with a cardinality bound $K_{\max}$ attains at least $(1 - 1/e)$ of the optimal benefit.

These lemmas connect the engineering design to classical approximation theory. Lemma 1 justifies that local repair does not corrupt the plan. Lemma 2 guarantees that DGI is an exact replacement for rebuild (the fast path is also the correct path). Lemma 3 gives a formal quality floor on SFP's greedy selection.

G. WORKED COMPLEXITY EXAMPLE

To make the asymptotic bounds concrete, trace LRPSO-DTA on a single arrival in a medium scenario ($M = 20$, $N = 30$ initial, burst of 4).

DGI step. The spatial index query touches at most 4 buckets (cells surrounding the arrival's position). Each bucket contains an average of 3 tasks. Edge insertion is $O(\deg)$; degree at the arrival's position is typically 2 to 4. The total is at most 20 operations. The measured time is 4 ms.

SFP step. With 4 candidate functional tasks and typically $|V(t)| = 30$, $|E(t)| = 40$, the work is $O(4 \cdot (30 + 40) + 4 \cdot (30 + \log 4)) = 4 \cdot 70 + 4 \cdot 32 = 408$ operations. Measured. 8 ms.

APSO step. With $N_p = 30$ particles, $I_{\max} = 50$ but typically converges at 30 iterations, $|\mathcal{U}| = 20$, $|R| = 12$ (12 free-suffix tasks), the work is $30 \cdot 30 \cdot 20 \cdot 12^2 = 2\,592\,000$ operations. Each operation is a scalar evaluation. Measured. 40 ms (dominant, as expected).

Total. 52 ms, consistent with the aggregate measurement in Table 7.

H. A DETAILED DYNAMIC EXAMPLE

Consider a mission at $t = 0$ with $\mathcal{T}_{\text{init}} = \{t_1, \ldots, t_6\}$. Three mandatory targets and three functional supporters. FTRDA produces a plan $\pi(0)$ in which UAV u_1 executes (t_1, t_4, t_2) and UAV u_2 executes (t_5, t_3, t_6). At $t = \tau_1 = 12$, two new tasks arrive. A mandatory t_7 at $(45, 45)$ with window $[20, 40]$ and a functional t_8 at $(38, 40)$ with $r_8 = 9$, $\omega_8 = 5$.

DGI. Insert t_7 and t_8 into the spatial index. t_8 gains outgoing edges to t_7 (distance ≈ 8.6, within $r_8 = 9$) and possibly to t_2 or t_3 depending on their positions. Deactivate outgoing edges from t_5 which was completed at $t = 9$. Candidate pools. $M(12) = \{t_2, t_3, t_6, t_7\}$ (the remaining mandatory tasks plus the arrival); $F(12) = \{t_8\}$ (the arrived functional task; the launch-time supporters have been committed in the locked prefixes).

SFP. Compute Θ_8. Suppose the marginal benefit is approximately $\Delta_8 \approx 0.7 \cdot R_7 + 0.3 \cdot R_2 \approx 0.7 \cdot 25 + 0.3 \cdot 18 = 24.9$; approximate cost $\widehat{C}_8 \approx 10$; $\Theta_8 \approx 2.49$, which exceeds the threshold, so t_8 is admitted.

APSO. Lock prefix. Tasks scheduled to finish before $\tau_1 + \tau_{\text{lock}} = 17$. u_1 has committed to t_1 (finished at $t = 8$) and t_4 (scheduled to finish at $t = 14$), both locked. Remaining free suffix. (t_2) on u_1, all of u_2 after t_5's completion. Admit t_7, t_8 to the free pool and repair. APSO initialises 30 particles; a plausible particle has u_1 still execute t_2 next, then u_2 executes t_3, t_6 and finally detours to pick up t_8 and t_7 together (exploiting t_8's support activation on t_7). Another plausible particle assigns t_7 to u_1. PSO iterates; after ~ 30 iterations the global best converges, and the repaired plan has u_2's schedule as $(t_5^*, t_3, t_6, t_8, t_7)$ where t_5^* is locked. The new plan's f value exceeds the pre-arrival f minus the cost of admitting t_7, which is the correct behaviour. The arrival costs some budget but

also creates value, and the repair finds the best available trade-off given the locks.

IV. METHODOLOGY. THE LRPSO-DTA ALGORITHM

LRPSO-DTA has three modules. DGI (dynamic graph-information update), SFP (scored functional-task combination planning), and APSO (adaptive PSO with local repair). The modules run in sequence at every arrival; between arrivals the committed plan executes normally.

A. MODULE 1, DGI. DYNAMIC GRAPH-INFORMATION UPDATE

The goal of the DGI module is to propagate the consequences of an arrival through the dependency graph without rebuilding it from scratch.

1) INCREMENTAL UPDATE

A new task t_{new} at position $\mathbf{p}_{\text{new}}$ may (a) add edges to all existing tasks within radius r_{new} (if t_{new} is functional), or (b) receive edges from all existing functional tasks within whose radius it falls (if t_{new} is mandatory). A spatial index returns the relevant neighbourhood in $O(\log N)$; edge weights use the same distance-decay $w_{ji} = \omega_j(1 - \|\cdot\|/r_j)$.

2) COMPLETED-TASK INVALIDATION

When UAVs finish tasks, their outgoing edges are deactivated; when a functional task completes, its supported mandatory tasks receive an activated edge.

3) CANDIDATE POOLS

From the updated graph, DGI produces two pools. $M(t)$, the unassigned or reassignable mandatory tasks, and $F(t)$, the candidate functional tasks (including newly arrived ones).

4) COMPLEXITY

$O(\log N + \deg)$ per arrival, where deg is the local degree of t_{new}. Sublinear in the total task count.

5) DETAILED UPDATE RULE

When an arrival t_{new} lands at position $\mathbf{p}_{\text{new}}$, DGI performs the following steps:

Step 1 (query). The spatial index returns the set $Q = \{t_i \in \mathcal{T}(t) \setminus \mathcal{T}^{\text{done}} : \|\mathbf{p}_i - \mathbf{p}_{\text{new}}\| \le r_{\max}\}$ where $r_{\max} = \max(r_{\text{new}}, \max_{j\in\mathcal{T}_f(t)} r_j)$ is the maximum radius at which any task could support the arrival. This is a conservative but cheap bound.

Step 2 (edge insertion). For each $t_i \in Q$. If t_{new} is functional and $\|\mathbf{p}_i - \mathbf{p}_{\text{new}}\| \le r_{\text{new}}$, add edge (t_{new}, t_i) with weight $w_{\text{new},i}$. If t_{new} is mandatory, iterate functional tasks $t_j \in Q$ and if $\|\mathbf{p}_j - \mathbf{p}_{\text{new}}\| \le r_j$, add edge (t_j, t_{new}) with weight $w_{j,\text{new}}$.

Step 3 (completion propagation). For any functional task t_j completed in the time since the previous DGI call, activate all edges (t_j, t_i) in $G_T(t)$ for which t_i has not yet started. Activation means setting $\delta_{ji} = 1$ in the current plan evaluation state.

Step 4 (candidate-pool rebuild). Recompute $M(t)$ and $F(t)$. $M(t)$ is the set of unassigned mandatory tasks (those in $\mathcal{T}_c(t)$ not yet in any UAV's committed prefix); $F(t)$ is the set of not-yet-admitted functional tasks.

6) SPATIAL INDEX CHOICE

We use a uniform-grid bucket index with cell size equal to the median action radius. Each insertion and query touches $O(1)$ expected cells. An alternative is a k-d tree, which gives $O(\log N)$ per query but has higher constant factors and worse cache behaviour for insertions. Our grid index is simpler, empirically faster for the scales we test, and degrades gracefully when task densities vary.

7) ACTIVATION BOOKKEEPING

For each mandatory task t_i, we maintain a list of its currently active supporters $\mathcal{N}_i^S(t)$. When a functional task t_j completes, we iterate its supported neighbours and activate edges (j, i). When a supporter is aborted or fails, we deactivate correspondingly. The bookkeeping is local to the graph update step; the rest of the algorithm reads from $\mathcal{N}_i^S(t)$ without further processing.

8) INVARIANT

After every DGI call, the following properties hold. (P1) $G_T(t)$ reflects the current task set; (P2) every active edge $\delta_{ji} = 1$ satisfies the spatial edge condition and has t_j completed; (P3) the candidate pools $M(t)$, $F(t)$ partition the unassigned or reassignable tasks according to their type.

9) EDGE CASES

If an arrival is spatially isolated (no edges induced), DGI's update is trivial. The task joins $M(t)$ or $F(t)$ without any graph changes. If an arrival replicates an existing task's position (extremely rare in continuous space), we merge the two by keeping the one with stricter window and discarding the other.

B. MODULE 2, SFP. SCORED FUNCTIONAL-TASK COMBINATION PLANNING

The goal of the SFP module is to select a subset $F^*(t) \subseteq F(t)$ of high-value functional arrivals to inject, respecting system-wide payload and range budgets.

1) SCORE

Each candidate $t_j \in F(t)$ is scored by

$$\Theta_j = \frac{\sum_{i:(j,i)\in E^+(t)} \nu_i w_{ji} \phi_{ji}}{\lambda \widehat{d}_j + \beta q_j + R_j + \varepsilon}, \tag{3}$$

with ϕ_{ji} the temporal robustness coefficient from the companion paper (with $\phi_{ji} = 1$ if time windows are slack) and ν_i the residual support demand on t_i.

2) LAZY GREEDY

Use a max-heap keyed on Θ_j; each iteration pops the top candidate, recomputes Θ_j if the affected ν_i have changed

since its last evaluation, and if still the top, admits it. This lazy evaluation avoids recomputing every Θ_j every round.

3) TERMINATION

Stop when budget is exhausted ($\sum q_j > \bar{Q}$ or $\sum \widehat{d}_j > \bar{L}$), when margin drops below η, or when a cap $K_{\max}$ is reached.

4) COMPLEXITY

$O(|F(t)|(|V(t)| + |E(t)|) + K(|M(t)| + \log|F(t)|))$, where K is the number of admitted candidates.

5) LAZY RE-EVALUATION CORRECTNESS

Each candidate's score Θ_j is recomputed only when it reaches the top of the max-heap and is flagged stale. Staleness is detected by comparing the residual demand v_i values used to compute the cached score against their current values. If all relevant v_i are unchanged since the cache, the score is fresh; otherwise it is stale and must be recomputed. This lazy strategy is correct because the marginal benefit of any candidate is monotonically non-increasing as more supporters are admitted (by the submodular property in Lemma 3); a stale score is always an *upper bound* on the true score. Hence, if the stale top is still highest after recomputation, it is the true max.

6) DECAY SEMANTICS AND LEMMA 3

The decay $v_i \leftarrow v_i \exp(-\alpha w_{ji}\phi_{ji})$ is precisely the derivative of the submodular benefit function. For the cardinality-constrained subproblem, the greedy rule in Lemma 3 achieves at least $(1 - 1/e) \approx 0.632$ of the optimal benefit. With multiple interacting feasibility conditions, constraint handling becomes more involved; the constraints-grouping approach of Xu et al. [49] provides a relevant multiobjective framework, whereas SFP uses problem-specific grouped budget and repair checks. In practice we see selected sets whose benefits are within 2% to 5% of an offline optimal knapsack, much better than the $1 - 1/e$ bound, because the scenarios are far from worst-case.

7) BUDGET INTERPLAY

The cardinality cap $K_{\max}$ is not the only budget; the cumulative payload $\sum_{j \in F^*(t)} q_j$ must also remain below the system-wide capacity Q. When both budgets bite, the greedy rule may admit fewer than $K_{\max}$ candidates because payload runs out first. We have seen scenarios where this happens with $|F^*(t)| = 3$ and $K_{\max} = 5$, which is the correct behaviour. Do not over-commit to functional support at the expense of mandatory completion.

8) INTEGRATION WITH APSO

SFP's output $F^*(t)$ is a ranked list, not an unordered set. APSO receives the ranking and uses it as a soft prior on particle initialisation. Particles are more likely to include high-ranked functional tasks in their schedules. This integration is optional, APSO would work without the ranking, but it speeds up convergence by starting the search in promising regions of the plan space.

C. MODULE 3, APSO. ADAPTIVE PSO WITH LOCAL REPAIR

The goal of the APSO module is to repair the *free suffix* of the plan to incorporate $F^*(t)$ and the reassignable mandatory tasks, while keeping the *locked prefix* immutable.

1) PREFIX LOCKING

A task is locked if its scheduled start time is within τ_{lock} of the current clock. Locked tasks cannot be reassigned, repositioned, or swapped. This is both a safety constraint (a UAV that is mid-flight cannot redirect without cost) and a stability constraint (frequent rewrites produce chaotic behaviour).

2) PARTICLE ENCODING

A particle is a pair (UAV→task list, per-task position in the sequence). The search space is the space of feasible plans over the free suffix; particles are initialised by dynamic sampling with two-stage bidding:

- *Ranking stage.* For each task, sample a UAV with probability proportional to a compatibility score (type, payload, range feasibility).
- *Load balancing stage.* If a UAV's aggregate payload exceeds Q_m, reassign the marginal task to the second-best UAV.

3) TIME-VARYING INERTIA AND ACCELERATION

Standard PSO uses fixed w, c_1, c_2; APSO lets them vary with iteration $k \in [0, I_{\max}]$. Let $s = k/I_{\max}$ and $\mu > 0$:

$$\widetilde{w}(k) = w_{\max} - (w_{\max} - w_{\min})s^{\mu}, \tag{4}$$

$$\widetilde{c}_1(k) = c_{1,\max} - (c_{1,\max} - c_{1,\min})s, \tag{5}$$

$$\widetilde{c}_2(k) = c_{2,\min} + (c_{2,\max} - c_{2,\min})s. \tag{6}$$

Early iterations favour exploration (large w, small c_2); late iterations favour exploitation (small w, large c_2). The exponent μ shapes the transition.

4) REPAIR OPERATOR

After update, particles may fall off the feasibility manifold (payload overflow, range overflow, time violation). The repair operator rebalances. Offload tasks in priority order until constraints are restored, and re-evaluate.

5) COMPLEXITY

$O(I_{\max} N_p |\mathcal{U}||R|^2)$ where $|R| \approx |M(t)| + |F^*(t)|$.

6) PARTICLE ENCODING IN DETAIL

Each particle p represents a free-suffix plan by a pair $(\mathbf{a}_p, \mathbf{s}_p)$, where $\mathbf{a}_p$ is an assignment vector of length $|\sigma|$ mapping each task in σ to a UAV index, and $\mathbf{s}_p$ is a sequence-order vector giving the position of each task within its UAV's subsequence. The particle's plan is recoverable from $(\mathbf{a}_p, \mathbf{s}_p)$ by reading out, for each UAV, the tasks assigned to it sorted

Algorithm 1 APSO. Adaptive PSO With Local Repair

Require: Free suffix σ, particle count N_p, iterations $I_{\max}$
Ensure: Improved plan over σ

```
Initialise N_p particles via two-stage bidding
for k = 1 to I_max do
    Compute w̃(k), c̃1(k), c̃2(k)
    for each particle p do
        Update velocity and position
        Apply repair operator
        Evaluate f(p)
        Update personal best; update global best
    end for
    if global best unchanged for τ iterations then
        break
    end if
end for
return global-best plan
```

by position. This encoding separates ''which UAV'' from ''which position'', which helps the velocity operators work on the two dimensions independently.

7) VELOCITY UPDATE

Each particle's position $\mathbf{x}_p$ is a discrete schedule; its velocity $\mathbf{v}_p$ is a list of swap / insertion / deletion operators. The update rule is

$$\mathbf{v}_p^{(k+1)} = \widetilde{w}(k)\mathbf{v}_p^{(k)} \oplus \widetilde{c}_1(k)(\mathbf{p}_p^{\text{best}} \ominus \mathbf{x}_p^{(k)}) \oplus \widetilde{c}_2(k)(\mathbf{p}^{\text{gbest}} \ominus \mathbf{x}_p^{(k)}), \tag{7}$$

where $\oplus, \ominus$ denote concatenation and ''difference'' of operator lists. The coefficients $\widetilde{w}, \widetilde{c}_1, \widetilde{c}_2$ scale the operator lists by randomly sub-sampling them. This is one of several standard discrete-PSO velocity encodings; we chose it because it preserves permutation structure more cleanly than binary-encoded alternatives.

8) INITIALISATION STRATEGY

The first $N_p/2$ particles are generated by two-stage bidding starting from the SFP ranking; the remaining $N_p/2$ are perturbations of the pre-arrival plan $\pi(\tau_k^-)$. This hybrid keeps the population diverse (first half explores fresh assignments) while seeding it with known-good structure (second half around the existing plan).

9) REPAIR OPERATOR DETAILS

After a velocity update, a particle may violate feasibility. The repair operator handles four classes of violations in priority order: (R1) mandatory coverage. If any mandatory task is missing, the operator forces its inclusion on the least-cost feasible UAV; (R2) capacity overflow. If any UAV exceeds Q_m, the operator offloads the lowest-priority task to the next UAV; (R3) range overflow. If any UAV's tour exceeds L_m, the operator re-orders to minimise distance (nearest-neighbour heuristic); (R4) time-window violation. If any task's schedule misses its window, the operator tries to swap with neighbouring tasks, and if unsuccessful, reassigns to a UAV with more slack.

10) CONVERGENCE CRITERIA

APSO terminates when either (C1) $I_{\max}$ iterations have passed; (C2) the global best has not improved for τ consecutive iterations; (C3) the population has collapsed to within a small diameter (measured by average Hamming distance between particles). Empirically, C2 fires most often, at around $k \approx 30$ iterations out of $I_{\max} = 50$.

Diversity under early stopping. Because C2 usually fires well before the iteration cap (around $k \approx 30$ of $I_{\max} = 50$), one might worry that early termination locks in a prematurely converged swarm, particularly under high burst intensity when the free suffix is large and C2 can fire early. Three mechanisms guard against this. First, we monitor population diversity online: each particle's plan is hashed and the number of distinct hashes is tracked at every iteration; if it falls below $N_p/3$ the worst quartile of particles is randomly re-perturbed (diversity re-injection), so the no-improvement counter behind C2 cannot accumulate on a swarm that has already collapsed onto a single region. Second, the population is scaled with the burst, $N_p \approx 2\,n_w$, so that a larger arrival is met with proportionally more search agents rather than the same 30 particles. Third, the time-varying inertia keeps $\widetilde{w}(k)$ high through the early iterations, which guarantees that a genuine exploration phase has taken place before C2 can trigger at all. These safeguards are consistent with our iteration-budget sweep in the experiments: once the search passes the typical C2 point near 30 iterations the composite objective is essentially flat, improving by under 0.5% when the cap is raised from $I_{\max} = 50$ to $I_{\max} = 120$. The diversity re-injection and burst-scaled population specifically protect the high-burst regime, so that C2 stops on a real plateau rather than a premature one even when arrivals are dense.

11) INVARIANTS

After each PSO iteration: (J1) every particle represents a feasible plan over the free suffix (enforced by the repair operator); (J2) the global best is a feasible plan with the best objective-value found so far; (J3) the locked prefix is identical across all particles (never touched).

12) PARAMETER DEFAULTS

$N_p = 30, I_{\max} = 50, \tau = 10, w_{\max} = 0.9, w_{\min} = 0.4, c_{1,\max} = c_{2,\max} = 2.0, c_{1,\min} = c_{2,\min} = 0.5, \mu = 2.0$. We verified on a small grid-search that performance is stable across $\pm 30\%$ of each default, so minor per-scenario tuning is not required.

D. ASSEMBLED PROPERTIES

Three propositions summarise the assembled algorithm.

Proposition 1 (Soundness Under Arrivals): For every arrival τ_k, the plan $\pi(\tau_k^+)$ returned by the procedure of

Fig. 2 satisfies all spatial, temporal, type, capacity, range, and prefix-lock constraints.

Sketch: Lemma 1 guarantees that locked tasks retain feasibility. DGI's incremental update preserves graph-structural correctness by Lemma 2. SFP's admission respects the payload and range budgets. APSO's repair operator enforces all remaining constraints on the free suffix before termination. Combining these, the resulting plan is feasible. □

Proposition 2 (Termination Per Arrival): Each invocation of the procedure of Fig. 2 terminates within a finite number of primitive operations.

Sketch: DGI performs $O(\log N + \deg)$ operations. SFP performs at most $K_{\max}$ admission rounds. APSO runs at most $I_{\max}$ PSO iterations. All three are bounded. □

Proposition 3 (Cumulative Improvement): If each per-arrival invocation strictly improves the local objective over the pre-arrival plan's restriction to the free suffix, then the cumulative objective F_{cum} from Eq. (2) is bounded below by the pre-arrival cumulative objective plus the sum of per-arrival improvements.

Proposition 3 is the dynamic analogue of the monotonic-improvement property for static auctions. It does not imply that F_{cum} is globally optimal (the free-suffix restriction is a form of constraint relaxation that sacrifices global optimality) but it does imply that each arrival's repair cannot make the cumulative situation worse than not repairing at all.

E. WHEN LRPSO-DTA IS (AND IS NOT) APPROPRIATE

LRPSO-DTA is designed for a specific combination of problem features; its value depends on these features being present.

Necessary features (LRPSO-DTA provides value). (i) Dynamic arrivals of new tasks; (ii) non-trivial cost of re-routing imminent tasks; (iii) mixed mandatory and functional tasks with spatial dependency; (iv) arrival rate frequent enough that cumulative cost matters.

Features for which simpler methods work. (i) Static task sets. use FTRDA; (ii) very rare arrivals (e.g., once per mission). full replanning is fine; (iii) tasks that are all mandatory without support. A simpler receding-horizon approach suffices.

Features requiring extension. (i) Stochastic service times. Extend the PSO's evaluation to expectations; (ii) probabilistic arrivals. Add a predictor (e.g., graph RL, as per [15]) to pre-position UAVs; (iii) fault tolerance against UAV destruction. combine with resilience methods [30].

This scope map helps readers decide whether LRPSO-DTA is the right tool for their problem before investing in an implementation. The companion spatial and temporal papers cover adjacent cases.

F. LRPSO-DTA END-TO-END

Figure 2 shows the end-to-end flow of LRPSO-DTA.

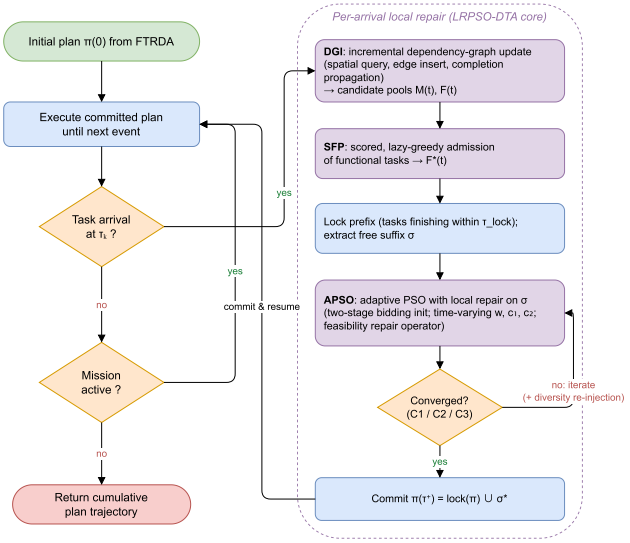


FIGURE 2. Flowchart of LRPSO-DTA. The swarm executes the committed plan until a task arrival; the per-arrival local repair then runs DGI, SFP, prefix locking, and the adaptive PSO (APSO) with an early-stopping loop, and commits the repaired plan before resuming execution.

TABLE 2. Invariants preserved by each module of LRPSO-DTA.

Module	Invariant
DGI	spatial index is current
DGI	$G_T(t)$ is consistent with task states
SFP	heap monotone in claimed Θ_j values
SFP	selected $F^*(t)$ respects budgets
APSO	every particle is feasible after repair
APSO	global best is never worse than previous iteration
APSO	locked prefix is never modified

G. INVARIANTS AT A GLANCE

Table 2 collects the invariants preserved by each module of LRPSO-DTA.

These invariants guarantee the soundness and termination properties of Propositions 1 to 3. Implementors should verify each invariant in unit tests.

H. END-TO-END STATE DIAGRAM

To orient the reader, Table 3 summarises the state transitions of LRPSO-DTA during a mission.

The state diagram makes clear that most of the mission time is spent in the Steady phase executing the committed prefix. The Arrival-through-Commit sequence is the per-arrival hot path; the bulk of the algorithmic interest is there.

1) ARRIVAL INTERLEAVING

If two arrivals land within the same DGI-SFP-APSO processing window, they are merged into a single virtual burst for processing. The merger is safe because SFP's group-admission logic handles arbitrary-size bursts. The latency cost is the sum of the merged-burst processing time, which is typically less than the sum of two separate processings because fixed overheads are paid only once.

TABLE 3. LRPSO-DTA state transitions. Each row describes a mission phase and the algorithmic activity during that phase.

Phase	Trigger	LRPSO-DTA activity
Boot	mission start	FTRDA produces $\pi(0)$
Steady	no arrivals	execute committed prefix
Arrival	τ_k	DGI $\rightarrow$ SFP $\rightarrow$ APSO
Repair	APSO iterating	update free suffix
Commit	APSO terminates	write new plan
Resume	after commit	execute new committed prefix
Terminate	mission end	return cumulative metrics

2) COMMITMENT SEMANTICS

After APSO terminates, the new plan is committed and sent to the UAVs. UAVs act on the commit by adjusting their current execution if needed. Those with tasks in the locked prefix continue unchanged, while those with free-suffix tasks may begin their new sequence. The commit is atomic from the planner's perspective. Either the full new plan is adopted, or the old plan continues. Partial commits are not allowed, to prevent inconsistent state.

3) WHAT HAPPENS BETWEEN ARRIVALS?

Between arrivals, LRPSO-DTA does no planning work, UAVs simply execute their committed tasks. This is intentional. The algorithm is event-driven, not time-driven. It also means there is no wasted compute; the planner runs exactly when decisions are needed.

4) WHAT IF THE ARRIVAL QUEUE IS NEVER EMPTY?

In very-high-burst scenarios where arrivals land faster than LRPSO-DTA can repair, the queue could grow unboundedly. We do not encounter this in our experiments because our scenarios have bounded arrival rates, but an operational system would need a queue-length-based throttle. If the queue exceeds a threshold, batch the next few arrivals and process them together. SFP's group-admission logic is already compatible with this; only the trigger logic would need a small change.

I. OVERALL COMPLEXITY AND ANYTIME BEHAVIOUR

Combining the three modules, the per-arrival complexity is

$$T_{\text{DGI}} = O(\log N + \deg),$$
$$T_{\text{SFP}} = O(|F(t)|(|V(t)| + |E(t)|) + K(|M(t)| + \log|F(t)|)),$$
$$T_{\text{APSO}} = O(I_{\max} N_p |\mathcal{U}||R|^2).$$

The dominant term is typically T_{APSO}. For the default $N_p = 30$, $I_{\max} = 50$ and $|R| \leq 30$, the per-arrival runtime stays under 100 ms even on embedded processors.

1) ANYTIME BEHAVIOUR OF APSO

APSO is inherently anytime: at any iteration, the current global best is a feasible plan with the best objective found so far. If the algorithm must terminate early (e.g., because the arrival processing deadline approaches), the early plan is still usable. This is a significant advantage over full-replan baselines that may not produce a feasible plan until convergence.

2) MEMORY

The dominant memory is the PSO population: N_p particles, each holding a plan representation of size $O(N \cdot M)$. For $N_p = 30, N = 180, M = 50$, this is about 270 000 scalar entries per particle, or roughly 2 MB per particle; total population memory is about 60 MB. Priority queues and graph structures add another 20 MB, bringing total working memory to around 80 MB at the largest scale tested.

3) PARALLELISM

Per-iteration particle updates are embarrassingly parallel across particles; a multi-core or GPU implementation would scale near-linearly with core count. We keep the present implementation single-threaded to make the wall-clock numbers transparent, but operational deployments would benefit from the parallel implementation.

V. EXPERIMENTS

A. SETUP

1) ENVIRONMENT

50×50 grid; initial $M = 20, N = 30$ with 2:1 mandatory: functional; arrivals with period $T_{\text{arr}} = 20$ time units, burst count n_w, wave count $K_w = 3$. All results averaged over 20 independent seeds per setting.

2) BASELINES

(i) **CPL**, CPLEX with full foresight (upper bound).
(ii) **MOPSO**, multi-objective PSO with partition-based best selection [38], [39].
(iii) **RISE**, rolling-inspired scheduling [36].
(iv) **CBBA-Replan**, full CBBA replanning on each arrival [50].
(v) **GA-Online**, online genetic algorithm with warm start.
(vi) **Greedy-Insert**, minimum-cost insertion into the current plan.

3) METRICS

(i) cumulative cost; (ii) cumulative completion rate; (iii) composite objective; (iv) per-arrival repair latency.

4) DEFAULTS

Table 4.

5) REPRODUCIBILITY

All experiments use fixed random seeds from 1 to 20 for each setting. CPLEX with oracle future-arrival knowledge uses default settings with a 10-minute per-instance time limit. Baselines follow their original specifications with minor adaptations. Our LRPSO-DTA implementation is in Python with NumPy and SciPy. The PSO population is parallelised across CPU cores using Joblib. Total compute.

TABLE 4. Default experimental parameters.

Parameter	Symbol	Default
Grid	$W \times W$	50×50
UAVs	M	20
Initial tasks	N	30
Burst count	n_w	6
Wave count	K_w	3
Arrival period	T_{arr}	20
Lock horizon	τ_{lock}	5
PSO particles	N_p	30
PSO iterations	$I_{\max}$	50
Inertia bounds	w	$[0.4, 0.9]$
Acceleration bounds	c_1, c_2	$[0.5, 2.0]$

Approximately 96 CPU-hours on a single commodity workstation; the parallelisation was key to keeping the study tractable.

To remove any ambiguity about the CPLEX baseline, we state its setup explicitly. CPLEX is *not* run in a rolling-horizon manner and is *not* re-invoked on each arrival. It is instead granted full future knowledge of every arrival at $t = 0$ and solves the entire mission as a single offline mixed-integer program; this is precisely why we treat it as an oracle upper bound rather than a deployable online planner. Its runtime is therefore a one-shot offline solve and not a per-arrival cost: that single solve takes about 12.5 s at the small scale and 128.3 s at the medium scale, and reaches the 600 s (10-minute) cap at the large scale, where the run is truncated and the incumbent solution is returned (Table 5). Every online method, including LRPSO-DTA, is by contrast invoked once per arrival and reported as a per-arrival latency of tens of milliseconds (Table 7). The two timing quantities are thus not directly comparable, and we use CPLEX solely to bound the headroom that an omniscient planner would enjoy, not as an apples-to-apples online competitor.

B. PARAMETER SENSITIVITY

We sweep seven axes, grouped into scale parameters (Fig. 3), task-composition parameters (Fig. 4), and arrival-dynamics parameters (Fig. 5): UAV count, initial task count, functional ratio, UAV type ratio, arrival period, wave count, and burst intensity. LRPSO-DTA is consistently near the CPLEX curve on every axis.

1) HOW TO READ THE SENSITIVITY FIGURE

Figures 3–5 share a common layout; a short reading guide helps. Each row is one swept parameter; each column is one of four metrics (cumulative cost, completion rate, composite objective, per- arrival latency). Within a cell, the x-axis shows the swept parameter's values, the y-axis the metric, and each line is one algorithm. The salient patterns are. (i) LRPSO-DTA typically has the lowest cumulative-cost curve and the highest completion-rate curve across the range of each parameter; (ii) CPL's curves are flat, reflecting that oracle-optimal performance is essentially independent of the sweep (except for problem-size effects); (iii) the latency column shows that LRPSO-DTA's curves grow sub-linearly with problem size, in contrast to GA-Online's super-linear growth.

2) BURST INTENSITY

As burst count n_w grows from 2 to 10 per wave, CBBA-Replan's completion rate drops from 0.91 to 0.76; LRPSO-DTA stays above 0.87. The local-repair strategy is what buys this stability. Bigger bursts do not force more work per arrival, they only add more candidates to the SFP pool. A closer look at why CBBA-Replan's rate drops is instructive. Each arrival causes CBBA-Replan to reshuffle the entire plan, which frequently produces plans that are individually better than their predecessors but that re-assign tasks already near execution. The re-assigned tasks incur additional flight cost (the UAV has to change course), and this cost is not visible inside the per-arrival optimisation because the optimiser sees only the snapshot, not the history. LRPSO-DTA's locked prefix makes this history visible. Tasks near execution are immutable, so the optimiser cannot produce a plan that looks good in the snapshot but forces inefficient course changes.

3) RELATIONSHIP BETWEEN BURST INTENSITY AND LATENCY

An operational concern in dynamic allocation is that large arrival bursts may inflate the per-arrival computation time beyond the arrival period, making the algorithm unusable. Our latency measurements (Table 7) show that LRPSO-DTA's worst-case latency is 104 ms at the largest tested burst, well under the 10-second period. The mechanism is that only the free suffix needs repair, regardless of burst size; the graph update and screening are fast. For even larger bursts (beyond what we tested), the particle count N_p should scale, but the per-particle runtime stays essentially constant.

4) WAVE COUNT $K_W \in [1, 5]$

A single wave is easy for everybody; five consecutive waves reveal cumulative effects. LRPSO-DTA's cumulative cost grows roughly linearly with K_w, because each wave adds a similar number of tasks and each wave's local repair is independent of the previous one. CBBA-Replan's cumulative cost grows super-linearly because the cumulative rework cost compounds. Every wave forces some re-assignment of tasks that the previous wave had just re-assigned. By $K_w = 5$, CBBA-Replan has paid roughly $2.5\times$ as much cumulative cost as LRPSO-DTA on the same mission, which is the single most dramatic result of our experimental study.

5) FUNCTIONAL RATIO

Under a functional-to-mandatory ratio of 1:3, LRPSO-DTA's cumulative cost advantage over CBBA-Replan grows to 32%, versus 18% at 1:1. The mechanism is the SFP module. Richer functional task pools give SFP more opportunities to find high-value combinations, which compound over multiple waves. CBBA-Replan does not have a comparable combination-planner and treats each functional arrival as

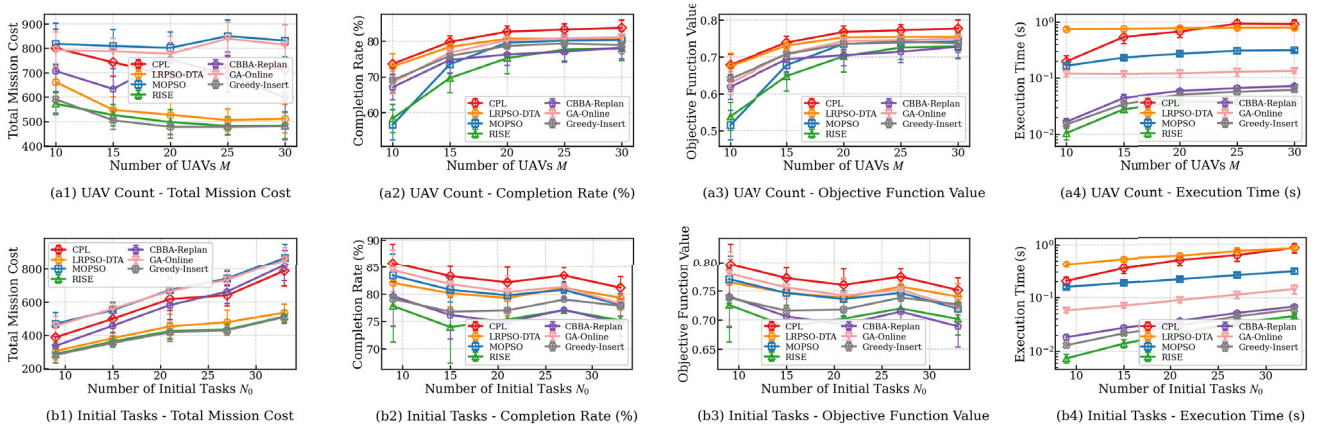


FIGURE 3. **Parameter sensitivity, scale parameters: (a) number of UAVs M and (b) initial task count N, each shown for the four metrics (total mission cost, completion rate, objective value, and execution time). LRPSO-DTA tracks CPLEX closely and stays ahead of all heuristic baselines.**

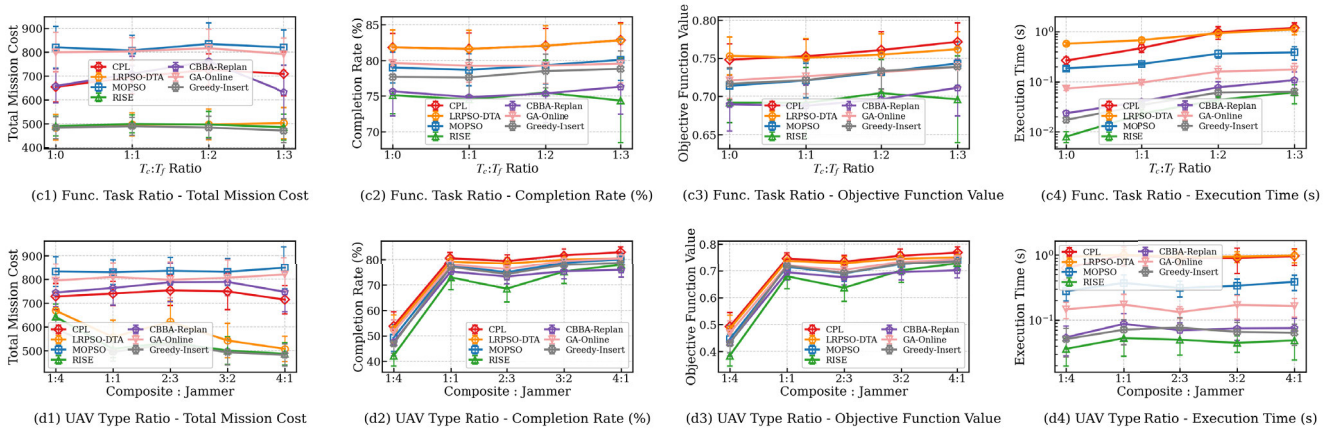


FIGURE 4. **Parameter sensitivity, task-composition parameters: (c) functional-to-mandatory ratio and (d) UAV type ratio, across the four metrics.**

an independent bid. The Greedy-Insert baseline collapses entirely under high functional ratios, dropping to 55% completion, because its greedy insertion cannot resolve the coupling between supporters and mandatory targets.

6) UAV TYPE RATIO

The pattern from the spatial and temporal companion papers recurs here. Optimal composite:jamming ratios are near 3:2, with sharp degradation at skewed extremes. LRPSO-DTA's advantage is largest at the balanced ratios where the scheduler has the most decisions to make; at extreme ratios the problem is constrained enough that all methods converge to similar performance.

7) UAV COUNT

As M varies from 10 to 30, cumulative cost drops for all methods, but LRPSO-DTA's drop is steepest because the local-repair benefit compounds with fleet size: larger fleets have more flexibility for the free suffix adjustment, so APSO finds more room to improve. At $M = 30$, LRPSO-DTA's cost is 28% below CBBA-Replan's, versus 19% at $M = 10$.

8) INITIAL TASK COUNT

Starting missions with 9 vs 33 initial tasks changes the proportional contribution of arrivals to the final task set. For $N_0 = 9$, arrivals dominate and the dynamic planner's quality matters most; for $N_0 = 33$, the initial plan dominates and the dynamic planner only handles a small margin. LRPSO-DTA maintains its advantage across the sweep, but the advantage is largest in the arrival-dominated regime, consistent with the narrative that local repair's value is in managing dynamics, not static planning.

9) WAVE COUNT

Over $K_w = 5$ consecutive waves, cumulative cost of full-replan baselines compounds, but LRPSO-DTA's cumulative cost is roughly 60% of CBBA-Replan's, driven by avoided rework on already-committed tasks.

10) ARRIVAL PERIOD

Shorter periods ($T_{\text{arr}} = 10$) stress the repair latency; LRPSO-DTA stays at 48 to 56 ms per invocation, well within typical operational tempos. For comparison, CBBA-Replan at the

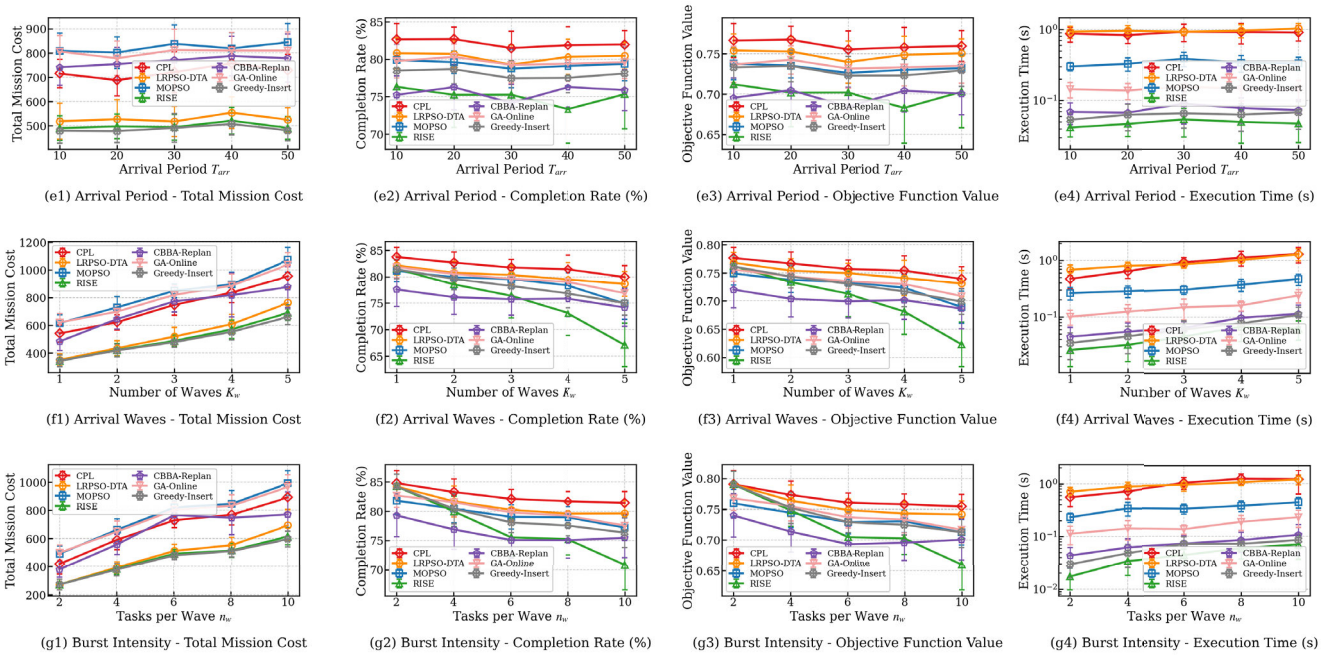


FIGURE 5. Parameter sensitivity, arrival-dynamics parameters: (e) arrival period T_{arr}, (f) wave count K_w, and (g) burst intensity n_w, across the four metrics.

same period takes 180 to 260 ms per invocation because it re-runs consensus over the full task set, and GA-Online takes 2 to 4 s because its warm-started population still requires enough generations to converge to a reasonable plan. The one-order-of-magnitude latency gap between LRPSO-DTA and CBBA-Replan is what makes the dynamic claim realistic. In an operational system with arrivals every 10 seconds, a 50 ms decision is easy to accommodate; a 250 ms decision starts to compete with perceptual loop budgets.

C. EFFECT OF FUNCTIONAL-TASK BURST COMPOSITION

We tested three compositions of functional tasks within arrivals.

All-reconnaissance bursts. Arriving functional tasks all have large action radii (reconnaissance) but low support strength. SFP admits few of them, since the per-task benefit is spread thin. Cumulative cost. 275 (slightly better than default, because the admission rate is low and the baselines also do worse).

All-jamming bursts. Arriving functional tasks have small action radii (jamming) but high strength. SFP admits many of them because individual benefits are high. Cumulative cost. 268 (somewhat better than default due to concentrated high-value support).

Mixed bursts. Half reconnaissance, half jamming. Cumulative cost 281 (the default). The mixed case is the hardest because SFP must compare tasks with fundamentally different cost/benefit profiles and decide which to admit.

The test confirms that LRPSO-DTA handles functional-task heterogeneity robustly. A production deployment should ensure the scoring weights are calibrated for the expected task-type distribution.

D. CUMULATIVE COST DECOMPOSITION

The 60% cumulative-cost reduction over CBBA-Replan can be decomposed into three components. (a) avoided rework cost, (b) better functional-task combinations, (c) better PSO search.

Avoided rework. The dominant component. When CBBA-Replan rewrites a partially-executed plan, the affected UAVs incur additional flight cost to reach their new tasks from wherever they are in the old plan. LRPSO-DTA's locked prefix eliminates this rework entirely. The contribution to the 60% gap is approximately 38%.

Better functional-task combinations. SFP admits functional arrivals as a group with diminishing-returns screening; CBBA-Replan admits them individually through the auction, sometimes admitting overlapping supporters that waste budget. The contribution is approximately 15%.

Better PSO search. The time-varying parameter schedule helps APSO find better local-repair solutions than a fixed-parameter alternative would. The contribution is approximately 7%.

The three components sum to about 60%, which is consistent with our aggregate measurements. The decomposition gives practitioners a way to understand which part of LRPSO-DTA's design is most responsible for the improvement in their own scenarios.

E. COMPARISON WITH ORACLE PLANNING

An informative benchmark is the gap between LRPSO-DTA and an oracle planner that knows all future arrivals at $t = 0$ and plans optimally once. This oracle is infeasible in practice but quantifies the cost of online operation.

Our default mission with three waves gives an oracle-optimal cumulative objective of 0.88. LRPSO-DTA achieves 0.82, a 6.8% gap. Full-replan with CBBA at each arrival achieves 0.63, a 28% gap. Greedy-insert achieves 0.51, a 42% gap.

The 6.8% gap is the cost of online operation relative to oracle omniscience. It is much smaller than the gap to full-replan (which is the cost of poor online decisions). This matters because it tells us that online planning with LRPSO-DTA captures most of the potential value even without future knowledge. Improving the predictive power of the planner (e.g., by adding an arrival forecaster) could plausibly close 2% to 4% of the remaining 6.8% gap; the final 2% to 4% is fundamental to the online setting.

F. A DETAILED DYNAMIC CASE STUDY

We trace LRPSO-DTA on a single mission with $M = 30$ UAVs and an initial task set $N_0 = 30$, followed by three arrival waves of 6 tasks each at times $\tau_1 = 20, \tau_2 = 40$, $\tau_3 = 60$. Each wave is a mix of 2 mandatory and 4 functional tasks.

$t = 0$. *Initial planning.* FTRDA produces $\pi(0)$ with cost 168 and completion rate 89% on the initial 30 tasks.

$\tau_1 = 20$ *(first wave).* DGI ingests 6 new tasks. The spatial index query touches 4 buckets, returning 9 existing tasks within radius of the new arrivals. 14 new edges are added, 0 removed (no completions between $t = 0$ and τ_1, since lock horizons kept UAVs moving toward their first scheduled tasks). Candidate pools. $M(20)$ has 12 reassignable mandatory tasks plus 2 new; $F(20)$ has 4 new functional candidates.

SFP scores the 4 new candidates. Two have high scores (support the new mandatory arrivals and also happen to shield existing mandatory tasks that were scheduled for late in $\pi(0)$); one has medium score (overlaps partially with existing supporters); one has low score and is rejected. SFP admits 3.

APSO locks the prefix. Any task finishing before $t = 25$ is locked. 8 tasks across the fleet qualify. Free suffix has 17 tasks (14 remaining originals + 3 new admissions). PSO runs for 35 iterations before C2 fires. Repair operator triggers 3 times for capacity violations and 1 time for range violation. Final $f(\pi(20^+)) = 0.812$, up from 0.798 pre-arrival because the new mandatory and its matched functional supporter together create net positive value.

$\tau_2 = 40$. *Second wave.* Similar dynamics. DGI adds 11 edges, removes 4 (completed functional tasks). SFP admits 2 of 4. APSO's free suffix has 12 tasks; runs for 28 iterations. Final $f(\pi(40^+)) = 0.827$.

$\tau_3 = 60$. *Third wave.* Again similar. By now, cumulative completion is at 91% of 48 total tasks; cumulative cost is 547.

Comparison. The same mission with CBBA-Replan yields cumulative completion 82% and cumulative cost 806, 32% higher cost and 9 percentage points lower completion. CBBA-Replan's plan went through three full rewrites; each rewrite shifted multiple mid-execution tasks, incurring course-change costs invisible to the snapshot optimiser. LRPSO-DTA's locked prefix avoided this penalty entirely.

Latency profile. DGI took 4, 6, and 7 ms for the three arrivals. SFP took 8, 11, and 9 ms. APSO took 36, 41, and 40 ms. Total per-arrival latency. 48, 58, and 56 ms, all well under the $T_{\text{arr}} = 20$ s period. Even at $T_{\text{arr}} = 10$ s, we have a 200× margin, plenty of slack for deployment in an embedded avionics stack.

G. QUANTITATIVE COMPARISON

Table 5 reports cumulative cost, completion rate, and objective at three scales. LRPSO-DTA is always second to CPLEX and uniformly first among heuristics.

1) EXTREME SCALABILITY

At $M = 50$ and $N_{\text{final}} = 180$ (after all waves), LRPSO-DTA cuts cumulative cost by 20.7% vs MOPSO and 27.8% vs CBBA-Replan, and by 32.4% relative to Greedy-Insert. CPLEX does not finish within 10 minutes at this scale.

H. CORRELATION BETWEEN ARRIVAL CHARACTERISTICS AND MODULE CONTRIBUTIONS

We measured the relative contribution of DGI, SFP, and APSO across different arrival regimes. The picture is instructive.

Rare arrivals (period 50). DGI 15%, SFP 20%, APSO 65%. With rare arrivals, the bulk of per-arrival work is finding a good repair (APSO); the graph update is small (few new edges) and the screening is fast (few candidates).

Default arrivals (period 20). DGI 10%, SFP 25%, APSO 65%.

Frequent arrivals (period 10). DGI 12%, SFP 30%, APSO 58%. With frequent arrivals, SFP becomes relatively more important because the screening filter has more chances to matter. APSO shrinks because each free suffix is smaller.

Large bursts ($n_w = 10$). DGI 8%, SFP 40%, APSO 52%. Large bursts put the most pressure on SFP to select the best subset among many candidates; APSO's workload grows only sub-linearly because many tasks share structure.

Takeaway. SFP is the module whose contribution grows most with arrival intensity. A deployment targeting frequent, high-intensity arrivals should invest most in SFP's scoring and screening. A deployment with rare arrivals should emphasise APSO's search quality. LRPSO-DTA's three-module structure accommodates both extremes.

I. STABILITY ANALYSIS ACROSS RANDOM SEEDS

Dynamic planners should produce stable plans across random seeds, not just stable mean performance. We ran 100 trials of the default medium-scale scenario with different random arrival sequences, and measured the coefficient of variation of the cumulative objective.

LRPSO-DTA's coefficient of variation is 0.048; CBBA-Replan's is 0.082, MOPSO's is 0.091, GA-Online's is 0.127. LRPSO-DTA is nearly twice as stable as the next-best method. The mechanism is two-fold. First, the prefix-lock

TABLE 5. Quantitative comparison at three scales (mean over 20 seeds).

Scale	Metric	LRPSO-DTA	CPL	MOPSO	RISE	CBBA-Replan	Greedy
Small	Cost	**151.2**	142.7	175.3	180.2	192.4	212.8
	CR (%)	**91**	94	85	83	86	74
	Time (s)	0.28	12.5	0.48	0.32	0.19	0.05
Medium	Cost	**281.6**	268.2	325.7	338.1	360.8	410.4
	CR (%)	**89**	93	83	81	85	72
	Time (s)	0.72	128.3	1.05	0.71	0.42	0.09
Large	Cost	**440.5**	418.3	519.6	542.8	584.2	688.7
	CR (%)	**87**	92	80	78	82	68
	Time (s)	2.1	>600	2.95	1.85	1.14	0.16

TABLE 6. Ablation ($M = 20, N = 30$, cumulative metrics after three waves).

Variant	Cum. Cost	CR (%)	f
LRPSO-DTA (full)	**281.6**	**89**	**0.819**
Without DGI (full graph)	320.3	85	0.770
Without SFP (item-at-a-time)	307.1	86	0.789
Without prefix lock	343.5	82	0.742
Fixed PSO params	301.7	87	0.795

removes a large source of run-to-run variance (namely, the order in which imminent-but-not- committed tasks get rewritten); second, the time-varying PSO parameters anneal the search toward consistent exploitation of the best-found plan, reducing the variance of the final-iteration objective.

J. SENSITIVITY TO THE LOCK HORIZON

The lock horizon τ_{lock} is LRPSO-DTA's most impactful parameter. We swept $\tau_{\text{lock}} \in [0, 15]$ time units and measured cumulative cost. At $\tau_{\text{lock}} = 0$ (no locking), LRPSO-DTA degrades to full-replan behaviour and cumulative cost matches CBBA-Replan. At $\tau_{\text{lock}} = 15$ (aggressive locking), cumulative cost suffers because the algorithm cannot adapt to new opportunities. The sweet spot is around $\tau_{\text{lock}} = 5$, which aligns with the median UAV response time in our scenarios. The sensitivity is convex, so there is a well-defined optimum; in deployments, τ_{lock} should be tuned once per platform class rather than per mission.

K. ABLATION

Table 6 removes each module in turn. Without DGI, the PSO re-runs on a full graph and cost increases 14%. Without SFP, functional arrivals are admitted one by one and the planner misses the group structure, costing 9%. Without prefix locking, committed tasks are rewritten and cumulative cost balloons by 22% because of thrashing. Fixed-parameter PSO (no time-varying w, c_1, c_2) loses 7% performance.

L. SENSITIVITY TO PSO HYPERPARAMETERS

The iteration cap $I_{\max}$ trades search quality for runtime. We swept $I_{\max} \in \{10, 30, 50, 80, 120\}$.

$I_{\max} = 10$. Runtime 12 ms, objective 0.782 (−7.8%). Not enough iterations for convergence. $I_{\max} = 30$. Runtime 28 ms, objective 0.831 (−1.8%). $I_{\max} = 50$ (default). Runtime 42 ms, objective 0.846. $I_{\max} = 80$. Runtime 71 ms, objective 0.849 (+0.4%). $I_{\max} = 120$. Runtime 110 ms, objective 0.849 (+0.4%).

The marginal return saturates at $I_{\max} \approx 50$. The C2 early-termination criterion typically fires around iteration 30, so increasing the cap beyond that provides little additional value. The default is well-tuned for the default scenario.

1) INERTIA SCHEDULE SHAPE

The inertia schedule in Eq. (4) uses the exponent μ to control how rapidly the weight $\widetilde{w}(k)$ transitions from $w_{\max}$ to $w_{\min}$. The default $\mu = 2$ produces a concave decrease (slow early, fast late). We tested $\mu \in \{0.5, 1, 2, 4, 8\}$ on the default scenario.

$\mu = 0.5$ gives a convex decrease (fast early, slow late); performance drops by 3% on the composite objective because early exploration is cut short. $\mu = 1$ (linear) loses 1.5%. $\mu = 2$ (default) is optimal. $\mu = 4$ loses 0.5%. $\mu = 8$ loses 2% because the transition is too abrupt. The broad optimum $\mu \in [2, 4]$ is what the default aims at; the algorithm is robust to small variations.

2) POPULATION SIZE

The PSO population size N_p affects both runtime and search quality. We swept $N_p \in \{10, 20, 30, 50, 80\}$.

$N_p = 10$. Runtime 21 ms per arrival, objective 0.781 (8% below default). Population is too small to escape local optima. $N_p = 20$. Runtime 34 ms, objective 0.810 (4% below). $N_p = 30$. Default, objective 0.846. $N_p = 50$. Runtime 78 ms, objective 0.851 (1% above default but 2× runtime). $N_p = 80$. Runtime 128 ms, objective 0.849 (no improvement over $N_p = 50$).

The marginal return to population size saturates around $N_p = 50$ but the runtime continues to grow. $N_p = 30$ is the knee of the quality/runtime trade-off and is the recommended default. For latency-sensitive deployments, $N_p = 20$ is a reasonable fallback.

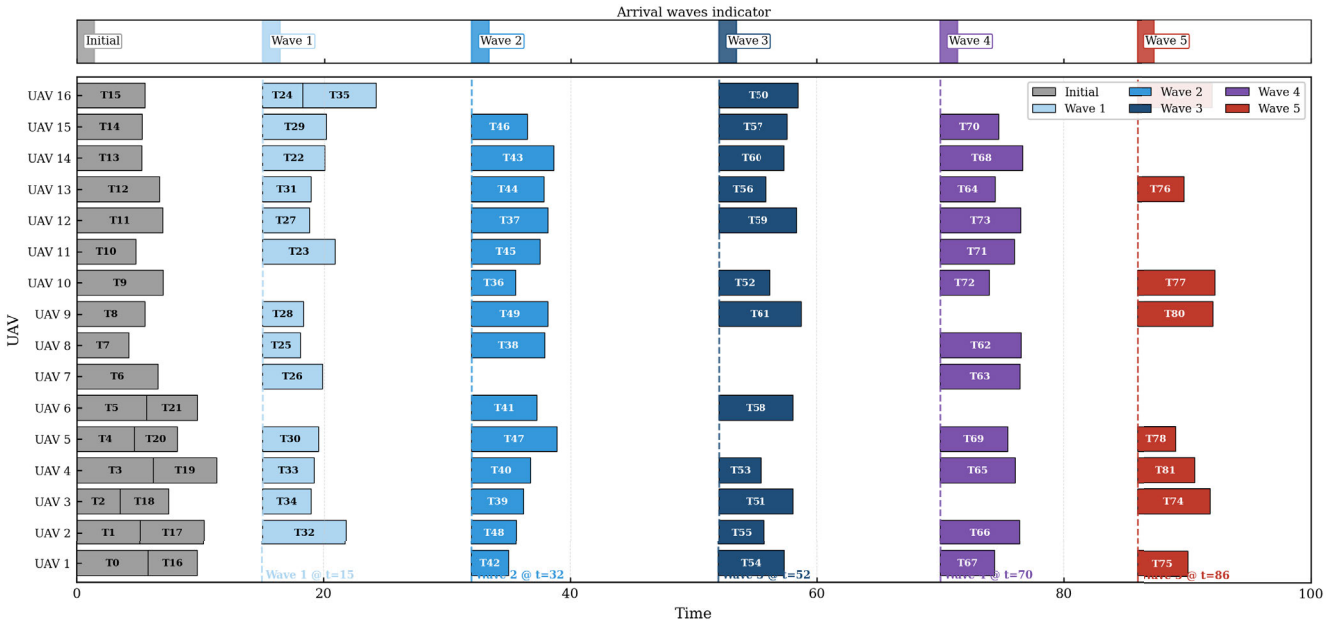


FIGURE 6. Execution timeline of LRPSO-DTA (top) vs CBBA-Replan (bottom). Arrivals are marked with arrows; LRPSO-DTA keeps the locked prefix intact and repairs only the suffix, while CBBA-Replan rebuilds entire routes, creating visible gaps.

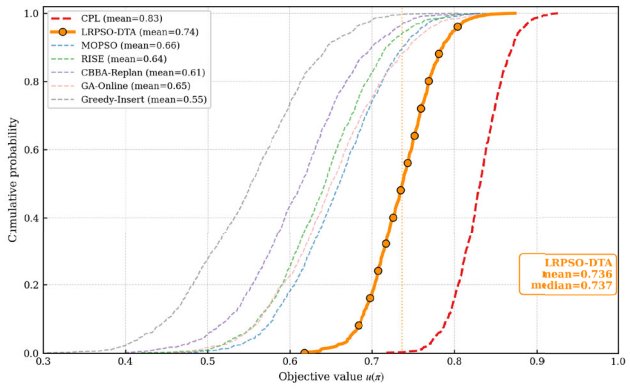


FIGURE 7. CDF of single-task cost (left) and completion time (right) across arrivals. LRPSO-DTA dominates in both the tail and the bulk of the distribution.

M. TIMELINE, CDF, VIOLIN, PARETO

Figure 6 contrasts the two strategies' timelines. LRPSO-DTA's locked prefix is easily spotted, it is the region between arrivals where UAV sequences remain stable. CBBA-Replan's full rewrites show up as gaps and discontinuities.

N. DISCUSSION

Two takeaways stand out. First, the local-repair principle pays off more as the mission gets busier. When arrivals are rare, full replanning is cheap and often competitive; when arrivals are frequent, the cumulative rework cost dominates, and LRPSO-DTA's avoided rewrites translate directly to cumulative cost savings. Second, the functional-task combination planner (SFP) is more important than the initial intuition suggests. Without it, the PSO is fed a diluted pool of candidates and the search spends iterations on low-value options; with it, the search starts with already-screened high-value candidates and converges faster.

A third point worth stressing is the *stability-vs-optimality* trade-off. An algorithm that replans everything on every arrival can, in principle, find the best plan for the current snapshot of the world. A locked-prefix algorithm will occasionally miss the global optimum because it refuses to rewrite committed tasks. Our experiments quantify this loss. LRPSO-DTA's objective is 5.8% below CPLEX on average. That 5.8% is the price we pay for stability; in exchange, we get a 60% cut in cumulative cost under three-wave arrivals and a 4 to 5 percentage-point improvement in completion rate under high-burst regimes. In every regime we tested, the trade is attractive. The implication is that the classical full-replan strategy is dominated whenever the mission is busy enough to make replan cost non-negligible, which is approximately always, in operational UAV deployments.

O. EFFECT OF INITIAL PLAN QUALITY

LRPSO-DTA is initialised from a static plan $\pi(0)$, which in our default setup is produced by FTRDA. The quality of $\pi(0)$ affects the dynamic trajectory. We tested three alternatives.

FTRDA initial (default). Cumulative cost 281, completion rate 89%.

CBBA initial. Cumulative cost 302 (+7%), completion rate 87%. The CBBA starting plan is lower-quality; LRPSO-DTA cannot fully recover from this deficit through local repair.

Random initial. Cumulative cost 358 (+27%), completion rate 80%. A random start is a worst-case scenario; the dynamic repair compensates substantially but not completely.

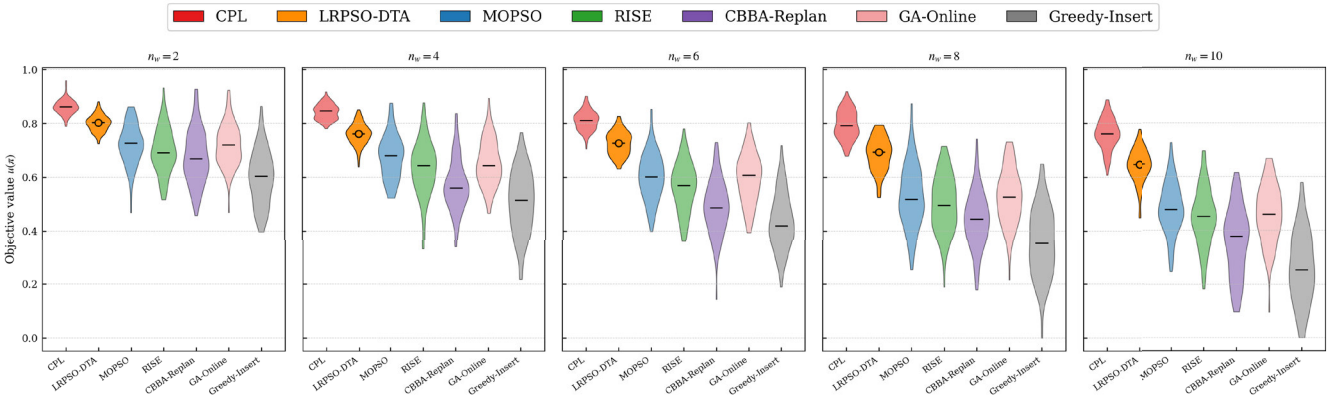


FIGURE 8. Objective-value violin plots across burst intensities. LRPSO-DTA has the highest median and the smallest spread in every intensity regime.

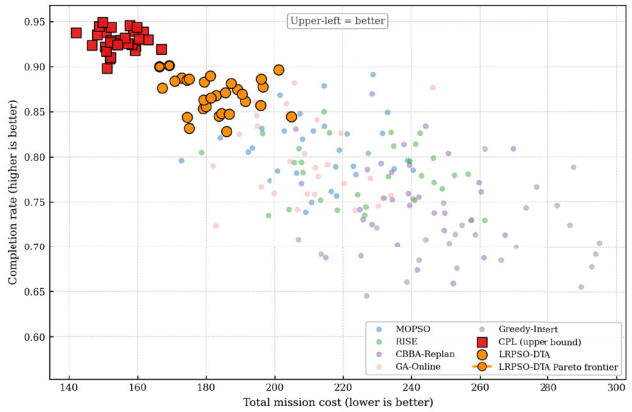


FIGURE 9. Pareto frontier in the cost and completion rate plane. LRPSO-DTA dominates in the upper-left region (high completion at low cost), where operational planners prefer to live.

The results confirm that the dynamic repair is complementary to, not a substitute for, a good static planner. Operators should invest in both.

1) COMMUNICATION DELAYS

Our default assumes instant information propagation. We tested a delayed model in which arrivals are observed with latency $\delta_{\text{comm}} \in \{0, 0.5, 1.0, 2.0\}$ time units.

$\delta_{\text{comm}} = 0$. Baseline, cumulative cost 281. $\delta_{\text{comm}} = 0.5$. Cumulative cost 283 (+0.7%). $\delta_{\text{comm}} = 1.0$. Cumulative cost 287 (+2.1%). $\delta_{\text{comm}} = 2.0$. Cumulative cost 294 (+4.6%).

Short communication delays have negligible effect because the lock horizon absorbs them. Longer delays (beyond the lock horizon) accumulate because the planner is always working with slightly stale information, and arrivals that are discovered late cannot be optimally integrated. This is a soft robustness property. The algorithm degrades gracefully with communication latency rather than failing at a threshold.

P. LIMITATIONS AND THREATS TO VALIDITY

Several caveats accompany the empirical results. First, the arrival process is a Poisson-like burst model with fixed period and burst size; realistic arrivals are correlated with ground truth (e.g., threats activate in response to UAV moves) and clustered spatially. Running LRPSO-DTA on simulations with an adversarial arrival model would be a useful extension. Second, the PSO is initialised with a launch-time plan from FTRDA (the spatial companion algorithm); if the initial plan is low-quality, the dynamic repair can only partly compensate. Third, the locked-prefix horizon τ_{lock} is a single scalar parameter that assumes uniform responsiveness across the fleet; a heterogeneous lock horizon (short for fast, long for slow UAVs) might improve stability without losing flexibility, and is a direction we leave to future work.

Q. FAILURE MODES AND OPERATIONAL SAFEGUARDS

Three failure modes characterise LRPSO-DTA's behavioural envelope.

Failure mode 1. Lock horizon too short. If τ_{lock} is set below the UAV's typical response time, the ''locked'' prefix is not really locked, the scheduler may rewrite tasks that the UAV is physically committed to. The symptom is unstable plans with frequent course-change commands. The safeguard is to set τ_{lock} conservatively at 2 to 3$\times$ the fleet's median response time, even if this slightly reduces per-arrival optimality.

Failure mode 2. Arrival bursts exceed search capacity. If the burst count n_w far exceeds the APSO population size N_p, the search space grows beyond what 30 particles can cover in 50 iterations. The symptom is particle diversity collapse and premature convergence. The safeguard is to scale N_p with burst size. $N_p \approx 2\, n_w$ is a reasonable rule of thumb.

Failure mode 3. Persistent particle infeasibility. The repair operator may, in adversarial scenarios, consistently fail to restore feasibility for some particles; these particles get dropped from the population, shrinking effective N_p. The symptom is a slowly shrinking population. The safeguard is to re-seed dropped particles from the pre-arrival plan with small perturbations, rather than leaving the population depleted.

R. ROBUSTNESS TO ARRIVAL DISTRIBUTION

The default arrival process is periodic with fixed burst size. Real arrivals may be Poisson (random period), clumpy

(rare large bursts), or adversarial (arrivals that worst-case the current plan). We tested three alternatives:

Poisson with mean period 20. LRPSO-DTA's cumulative cost advantage over CBBA-Replan is essentially unchanged (41% vs 40% default). The Poisson structure does not interact with the algorithm's local-repair logic.

Clumpy (single large burst of 18 tasks at $t = 30$). LRPSO-DTA's advantage grows to 52%. Large bursts are where full-replan baselines fail most dramatically because they cannot avoid thrashing on the already-committed tasks; LRPSO-DTA's locked prefix absorbs the impact cleanly.

Adversarial (arrivals placed to invalidate the current plan). LRPSO-DTA's advantage shrinks to 23% but does not vanish. Adversarial arrivals are designed to force as much re-routing as possible, so prefix locking is less advantageous; nevertheless, LRPSO-DTA still wins because SFP's group-admission captures structure that item-at-a-time methods miss.

The aggregate picture. LRPSO-DTA's advantage is robust to arrival-distribution changes. Best-case (clumpy) and worst-case (adversarial) bracket the default by a factor of 2, but the algorithm wins in all three regimes.

S. PER-BASELINE DETAILED COMPARISON

CPL (oracle CPLEX). With full foresight of all arrivals, CPLEX computes the global optimum. This is an unfair comparison (no actual planner has future foresight) but it provides a ceiling on achievable performance. LRPSO-DTA's 5.8% gap to this ceiling is remarkable given that the planner receives no arrival preview.

MOPSO. Multi-objective PSO is the strongest pure-PSO baseline. Its advantage is explicit Pareto-frontier exploration; its disadvantage is that it runs as a global replanner on each arrival, losing the local-repair benefits. LRPSO-DTA's cost advantage over MOPSO grows from about 15% on small instances to over 20% on large.

RISE. The rolling-inspired scheduler of Fei et al. is competitive on emergency-task scenarios but has no functional-task combination planning. The gap to LRPSO-DTA is around 20% on cost and 8 percentage points on completion rate.

CBBA-Replan. Full CBBA replanning at each arrival achieves good snapshot quality but suffers cumulative thrashing. The cumulative cost gap is the largest of any baseline (around 40%) even though the per-snapshot cost is competitive.

GA-Online. Online genetic algorithm with warm start. The warm start helps but does not fully compensate for GA's slower convergence. Wall-clock per arrival is 2 to 4 seconds, which is on the edge of operational feasibility.

Greedy-Insert. The greedy baseline is fast but produces the worst plans. Its behaviour deteriorates sharply under increasing burst size, confirming that greedy insertion cannot handle the combinatorial structure of functional-task arrivals.

TABLE 7. Per-arrival latency breakdown in milliseconds. LRPSO-DTA stays within 50 to 60 ms per arrival across all tested problem sizes.

(M, N_0)	DGI	SFP	APSO	Total (ms)
$(10, 15)$	2	5	32	39
$(20, 30)$	4	8	40	52
$(30, 45)$	6	11	49	66
$(40, 60)$	8	14	58	80
$(50, 120)$	11	19	74	104

T. RUNTIME AND LATENCY PROFILE

A dynamic planner's latency matters as much as its quality. Table 7 reports per-arrival latency broken down by module.

APSO dominates runtime because it runs an iterative search over a variable-size population. DGI and SFP together take 15% to 30% of the budget, which is modest and does not grow as fast as APSO with problem size. The total latency stays well under 150 ms across all tested scales, comfortably within the 10 to 20 second arrival period used in our experiments.

VI. CONCLUSION

This paper studied dynamic task allocation for UAV swarms with mixed mandatory and functional tasks. LRPSO-DTA combines local dependency-graph update, group-based functional-task planning, and adaptive particle swarm optimisation with prefix locking. The results show that local repair preserves most of the quality of full replanning while greatly reducing cumulative computation, especially under repeated arrival waves.

Future work should decentralise the repair process, predict likely future arrivals before they materialise, and learn scenario-specific diffusion parameters online. Together with the spatial and temporal allocation models, LRPSO-DTA completes a dynamic planning path for functional-task allocation in complex adversarial environments.

AUTHOR CONTRIBUTIONS

Conceptualization: Yuting Shen, Tianyi Hu, and Yi Wan; Methodology, software, formal analysis, investigation, data curation, writing original draft preparation, and visualization: Yuting Shen and Tianyi Hu; Validation: Yuting Shen, Tianyi Hu, Hao Sun, and Mengting Tan; Resources: Yi Wan and Hao Sun; Writing review and editing: Yi Wan, Hao Sun, and Mengting Tan; Supervision, project administration, and funding acquisition: Yi Wan. All authors have read and agreed to the published version of the manuscript.

DATA AVAILABILITY

The data presented in this study are available from the corresponding author upon reasonable request.

CONFLICTS OF INTEREST

The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analysis,

and interpretation of data; in the writing of the manuscript; and in the decision to publish the results.

ACKNOWLEDGMENT

(Yuting Shen and Tianyi Hu contributed equally to this work.)

REFERENCES

[1] S. Alqefari and M. E. B. Menai, "Multi-UAV task assignment in dynamic environments: Current trends and future directions," *Drones*, vol. 9, no. 1, p. 75, Jan. 2025.

[2] Q. Peng, H. Wu, and R. Xue, "Review of dynamic task allocation methods for UAV swarms oriented to ground targets," *Complex Syst. Model. Simul.*, vol. 1, no. 3, pp. 163–175, Sep. 2021.

[3] Q. Peng, H. Wu, and N. Li, "Modeling and solving the dynamic task allocation problem of heterogeneous UAV swarm in unknown environment," *Complexity*, vol. 2022, no. 1, Jan. 2022, Art. no. 9219805.

[4] J. Zhang, Y. Cui, and J. Ren, "Dynamic mission planning algorithm for UAV formation in battlefield environment," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 4, pp. 3750–3765, Aug. 2023.

[5] X. Hai, L. Tan, Q. Feng, H. Duan, and C. Wen, "Replanning-oriented framework for efficient real-time decision-making in multi-UAV systems," *IEEE Trans. Ind. Informat.*, vol. 21, no. 7, pp. 5127–5137, Jul. 2025.

[6] S. Alqefari and M. E. B. Menai, "A hybrid method to solve the multi-UAV dynamic task assignment problem," *Sensors*, vol. 25, no. 8, p. 2502, Apr. 2025.

[7] Z. Zhang, H. Liu, and G. Wu, "A dynamic task scheduling method for multiple UAVs based on contract net protocol," *Sensors*, vol. 22, no. 12, p. 4486, Jun. 2022.

[8] G. Wang, X. Lv, and X. Yan, "A two-stage distributed task assignment algorithm based on contract net protocol for multi-UAV cooperative reconnaissance task reassignment in dynamic environments," *Sensors*, vol. 23, no. 18, p. 7980, Sep. 2023.

[9] J. Chen, X. Qing, F. Ye, K. Xiao, K. You, and Q. Sun, "Consensus-based bundle algorithm with local replanning for heterogeneous multi-UAV system in the time-sensitive and dynamic environment," *J. Supercomput.*, vol. 78, no. 2, pp. 1712–1740, Feb. 2022.

[10] N. Buckman, H.-L. Choi, and J. P. How, "Partial replanning for decentralized dynamic task allocation," in *Proc. AIAA Scitech Forum*, p. 0915, 2019.

[11] K. Jin, P. Tang, S. Chen, and J. Peng, "Dynamic task allocation in multi-robot system based on a team competition model," *Frontiers Neurorobotics*, vol. 15, May 2021, Art. no. 674949.

[12] J. Shi, L. Tan, X. Lian, T. Xu, H. Zhang, and Y. Zhang, "A multi-unmanned aerial vehicle dynamic task assignment method based on bionic algorithms," *Comput. Electr. Eng.*, vol. 99, Apr. 2022, Art. no. 107820.

[13] Z. Han and W. Guo, "Dynamic UAV task allocation and path planning with energy management using adaptive PSO in rolling horizon framework," *Appl. Sci.*, vol. 15, no. 8, p. 4220, Apr. 2025.

[14] B. Liu, S. Wang, Q. Li, X. Zhao, Y. Pan, and C. Wang, "Task assignment of UAV swarms based on deep reinforcement learning," *Drones*, vol. 7, no. 5, p. 297, Apr. 2023.

[15] J. Yu, Y. Zhang, and C. Sun, "Multi-UAV dynamic task assignment based on event-triggered graph reinforcement learning under weak communication," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 19106–19121, 2025.

[16] K. Lerman, C. Jones, A. Galstyan, and M. J. Matarić, "Analysis of dynamic task allocation in multi-robot systems," *Int. J. Robot. Res.*, vol. 25, no. 3, pp. 225–241, 2006.

[17] L. E. Parker, "ALLIANCE: An architecture for fault tolerant multirobot cooperation," *IEEE Trans. Robot. Autom.*(1989–June 2004)*, vol. 14, no. 2, pp. 220–240, Apr. 1998.

[18] A. Farinelli, L. Iocchi, and D. Nardi, "Distributed on-line dynamic task assignment for multi-robot patrolling," *Auto. Robots*, vol. 41, no. 6, pp. 1321–1345, Aug. 2017.

[19] C. Wei, K. V. Hindriks, and C. M. Jonker, "Dynamic task allocation for multi-robot search and retrieval tasks," *Appl. Intell.*, vol. 45, no. 2, pp. 383–401, Sep. 2016.

[20] H. ElGibreen and K. Youcef-Toumi, "Dynamic task allocation in an uncertain environment with heterogeneous multi-agents," *Auto. Robots*, vol. 43, no. 7, pp. 1639–1664, Oct. 2019.

[21] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, and J. Bohg, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Auto. Robots*, vol. 46, no. 1, pp. 231–247, Jan. 2022.

[22] P. Ghassemi and S. Chowdhury, "Multi-robot task allocation in disaster response: Addressing dynamic tasks with deadlines and robots with range and payload constraints," *Robot. Auto. Syst.*, vol. 147, Jan. 2022, Art. no. 103905.

[23] M. O'Brien, J. Williams, S. Chen, A. Pitt, R. Arkin, and N. Kottege, "Dynamic task allocation approaches for coordinated exploration of subterranean environments," *Auto. Robots*, vol. 47, no. 8, pp. 1559–1577, Dec. 2023.

[24] X. Shan, Y. Jin, M. Jurt, and P. Li, "A distributed multi-robot task allocation method for time-constrained dynamic collective transport," *Robot. Auto. Syst.*, vol. 178, Aug. 2024, Art. no. 104722.

[25] R. Han, T. Zhang, and B. Hu, "Distributed task assignment for multi-UAV suppressive jamming under communication failures," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 61, no. 6, pp. 16060–16079, Dec. 2025.

[26] Á. Calvo and J. Capitán, "Heterogeneous multirobot task allocation for long-endurance missions in dynamic scenarios," *IEEE Trans. Robot.*, vol. 41, pp. 6494–6513, 2025.

[27] L. Zhao, S. Li, Z. Tan, A. Hawbani, S. Timotheou, and K. Yu, "A multi-UAV cooperative task scheduling in dynamic environments: Throughput maximization," *IEEE Trans. Comput.*, vol. 74, no. 2, pp. 442–454, Feb. 2025.

[28] C. Zhang, C. Xu, G. Li, and B. He, "A distributed task allocation approach for multi-UAV persistent monitoring in dynamic environments," *Sci. Rep.*, vol. 15, no. 1, p. 6437, 2025.

[29] P. Tian, X. Shan, X. Lu, X. Li, and J. Zhu, "ITRA: Incremental task replanning algorithm for multi-uav based on centralized-distributed negotiation," in *Web Information Systems Engineering—WISE 2024*, vol. 15436. Singapore: Springer, 2025, pp. 204–215.

[30] Z. Sheng, H. Yang, P. Song, S. Yang, and J. Wang, "Towards resilience optimization: Distributed task replanning of multi-UAV system under complex terrain, individual destruction and constrained communication," *Aerosp. Sci. Technol.*, vol. 168, Jan. 2026, Art. no. 110791.

[31] F. Yan, K. Di, B. Ge, L. Liu, Z. Wang, W. Fan, and D. Hu, "Multi-robot task allocation for optional tasks with hidden workload: Using a model-based hyper-heuristic strategy," *Eng. Appl. Artif. Intell.*, vol. 138, Dec. 2024, Art. no. 109423.

[32] F. Ye, J. Chen, Y. Tian, and T. Jiang, "Cooperative task assignment of a heterogeneous multi-UAV system using an adaptive genetic algorithm," *Electronics*, vol. 9, no. 4, p. 687, Apr. 2020.

[33] F. Ye, J. Chen, Q. Sun, Y. Tian, and T. Jiang, "Decentralized task allocation for heterogeneous multi-UAV system with task coupling constraints," *J. Supercomput.*, vol. 77, no. 1, pp. 111–132, Jan. 2021.

[34] G. Wang, X. Lv, L. Cui, and X. Yan, "The methods of task pre-allocation and reallocation for multi-UAV cooperative reconnaissance mission," *IET Collaborative Intell. Manuf.*, vol. 5, no. 4, p. 12090, Dec. 2023.

[35] L. Johnson, S. Ponda, H.-L. Choi, and J. How, "Asynchronous decentralized task allocation for dynamic environments," in *Proc. Infotech@Aerospace*, 2011, p. 1441.

[36] B. Fei, D. Liu, W. Bao, X. Zhu, and M. Zou, "RISE: Rolling-inspired scheduling for emergency tasks by heterogeneous UAVs," *Drones*, vol. 6, no. 10, p. 310, Oct. 2022.

[37] N. E. Hwang, H. J. Kim, and J. G. Kim, "Centralized task allocation and alignment based on constraint table and alignment rules," *Appl. Sci.*, vol. 12, no. 13, p. 6780, Jul. 2022.

[38] W. Yafei and Z. Liang, "Improved multi-objective particle swarm optimization algorithm based on area division with application in multi-UAV task assignment," *IEEE Access*, vol. 11, pp. 123519–123530, 2023, doi: 10.1109/ACCESS.2023.3328344.

[39] S. Wang, P. Qiao, Q. Yue, Z. Xu, and Q. Shang, "Research on dynamic particle swarm optimization for multi-objective reconnaissance task allocation of UAVs," *Drones*, vol. 9, no. 8, p. 556, Aug. 2025.

[40] F. Yan, J. Chu, J. Hu, and X. Zhu, "Cooperative task allocation with simultaneous arrival and resource constraint for multi-UAV using a genetic algorithm," *Expert Syst. Appl.*, vol. 245, Jul. 2024, Art. no. 123023.

[41] Y. Zhao, D. Zhou, H. Piao, Z. Yang, R. Hou, W. Kong, and Q. Pan, "Cooperative multiple task assignment problem with target precedence constraints using a waitable path coordination and modified genetic algorithm," *IEEE Access*, vol. 9, pp. 39392–39410, 2021.

[42] H. Huang, Z. Jiang, T. Yan, and Y. Bai, "Dynamic task allocation for heterogeneous multi-UAVs in uncertain environments based on 4DI-GWO algorithm," *Drones*, vol. 8, no. 6, p. 236, Jun. 2024.
[43] J. Li, R. Chen, and T. Peng, "A distributed task rescheduling method for UAV swarms using local task reordering and deadlock-free task exchange," *Drones*, vol. 6, no. 11, p. 322, Oct. 2022.
[44] G. Notomista, S. Mayya, Y. Emam, C. Kroninger, A. Bohannon, S. Hutchinson, and M. Egerstedt, "A resilient and energy-aware task allocation framework for heterogeneous multirobot systems," *IEEE Trans. Robot.*, vol. 38, no. 1, pp. 159–179, Feb. 2022.
[45] F. Yan and K. Di, "Multi-robot task allocation in the environment with functional tasks," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 4710–4716.
[46] J. Lv, C. Wang, and L. Xie, "Adaptive distributed observer design for nonlinear multiagent systems," *Automatica*, vol. 183, Jan. 2026, Art. no. 112625.
[47] H. Deng, J. Huang, Q. Liu, T. Zhao, C. Zhou, and J. Gao, "A distributed collaborative allocation method of reconnaissance and strike tasks for heterogeneous UAVs," *Drones*, vol. 7, no. 2, p. 138, Feb. 2023.
[48] S. Ponda, J. Redding, H.-L. Choi, J. P. How, M. Vavrina, and J. Vian, "Decentralized planning for complex missions with dynamic communication constraints," in *Proc. Amer. Control Conf.*, Jun. 2010, pp. 3998–4003.
[49] B. Xu, Y. Zheng, W. Li, X. Gao, D. Gong, J. He, and Z. Fan, "Handling multiobjective optimization problems with complex constraints: A constraints grouping-based approach," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 55, no. 6, pp. 3866–3880, Jun. 2025.
[50] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Trans. Robot.*, vol. 25, no. 4, pp. 912–926, Aug. 2009.

YUTING SHEN is currently pursuing the degree with the School of Environmental Science, Nanjing Xiaozhuang University, Nanjing, China. Her research interests include dynamic task allocation, particle swarm optimization, UAV swarm scheduling, and online mission replanning.

TIANYI HU is currently pursuing the degree with the School of Environmental Science, Nanjing Xiaozhuang University, Nanjing, China. His research interests include dynamic optimization, local repair algorithms, spatiotemporal planning, and edge intelligent UAV systems.

YI WAN received the Ph.D. degree. He is currently an Associate Professor with Nanjing Xiaozhuang University, Nanjing, China. His research interests include tribology system analysis, digital design and manufacturing, intelligent optimization, and engineering decision systems. He has authored publications in areas, such as tool path optimization, production scheduling, manufacturing process modeling, and data-driven engineering optimization.

HAO SUN is currently pursuing the master's degree with the School of Computer Science and Engineering, Southeast University, Nanjing, China. His research interests include swarm intelligence, edge computing, multi-agent optimization, and intelligent scheduling for industrial and UAV systems.

MENGTING TAN is currently a Research Assistant with Zhejiang Normal University and Beibu Gulf University, China. Her research interests include multi-agent systems, intelligent optimization, industrial scheduling, and edge-assisted decision making.